

UNIT - I

1

Formal Language Theory and Finite Automata

Syllabus

Finite Automata (FA) : An informal picture of FA, Finite State Machine (FSM), Language accepted by FA, Definition of Regular Language.

FA without output : Deterministic and Nondeterministic FA (DFA and NFA), epsilon- NFA and inter-conversion. Minimization of DFAs.

FA with output : Moore and Mealy machines - Definition, models, inter-conversion.

Contents

Part I : Finite Automata (FA)

- 1.1 Introduction to Formal Language
- 1.2 Alphabets and Languages Dec.-10,19, Marks 4
- 1.3 An informal Picture of FA
- 1.4 Representation
- 1.5 Finite State Machine (FSM)
- 1.6 Language Accepted by FA
- 1.7 Definition of Regular Language Dec.-18, Marks 3

Part II : FA without Output

- 1.8 Types of Finite Automata
- 1.9 Deterministic Finite Automata. May-10,12,13,18,19,
..... Dec.-14,16,17,18,
..... Aug.-15,17, Oct.-16, Marks 10
- 1.10 Non Deterministic Finite Automata Dec.-12,13,14, May-12,14,
..... Aug.-15,17, Marks 6
- 1.11 Epsilon- NFA Dec.-13,18,19, Aug.-17,
..... May-11, Marks 5

- 1.12 NFA with ϵ to NFA without ϵ **Dec.-14,15, Oct.-16,**
 **May-19,** Marks 6
- 1.13 NFA to DFA Conversion **Dec.-05,09,13,17,**
 **May-09,10,12,**
 **Aug.-15,17,** Marks 10
- 1.14 NFA with Epsilon to DFA Conversion **Dec.-11,** Marks 6
- 1.15 Minimization of DFAs **Dec.-12,** Marks 8

Part III : FA with Output

- 1.16 Moore and Mealy Machines **May-07,10,11,13,14,15,**
 **Dec.-06,11,13,14,19,**
 **Oct.-16,** Marks 10
- 1.17 Inter - Conversion **Dec.-05,11,12,18,**
 **May-06,07,12,14,19,**
 **Aug.-15,17, Oct.-19,** Marks 8
- 1.18 Case Study
- 1.19 Fill in the Blanks
- 1.20 Multiple Choice Questions

Part I : Finite Automata (FA)

1.1 Introduction to Formal Language

- Formal language is normally defined by using alphabets and formation rules.
- The **alphabet** of formal language is a set of symbols using which the language is built.
- The **formation rules** specify which strings of symbols are used in well-formed manner. The well-formed strings of symbols are also called **words**. The formation rules can be recursive.
- In theory of computations the formal languages are denoted by **regular expressions, regular languages, non regular languages, context free grammar** and so on.

1.2 Alphabets and Languages

SPPU : Dec.-10,19, Marks 4

1.2.1 Defining Languages

The language is a **collection of appropriate strings**.

The language is defined using an input set. The input set is typically denoted by letter Σ .

For example : $\Sigma = \{\epsilon, 0, 00, 000, \dots\}$

Here the language L is defined as 'any number of Zero's'.

Note that language is formed by appropriate strings and strings are formed by alphabets !

1.2.2 Symbols / Alphabets

An alphabet is a **finite collection of symbols**. We cannot define symbol formally.

The example of alphabet is,

$$S = \{a, b, c, \dots, z\}$$

The elements a, b, ..., z are the alphabets.

$$W = \{0, 1\}$$

Here 0 and 1 are alphabets, denoted by W.

Symbols : Members of an alphabet are called symbols.

For example, in string "0011" the prefixes can be 0, 00, 001. Similarly suffixes can be 1, 11, 011.

In string "Mango" the prefix can be 'Man' and the suffix can be 'go'.

1.2.3 String or Word

String is finite collection of symbols from alphabet.

For example, if $\Sigma = \{a, b\}$ then various strings that can be formed from Σ are {ab, ab, aa, aaa, bb, bbb, ba, aba ...}. An infinite number of strings can be generated from this set.

- The **empty string** can be denoted by ϵ .
- The **prefix** of any string is any number of leading symbols of that string and **suffix** is any number of trailing.

Review Questions

1. Define the following terms with example - 1) Symbol 2) Alphabet.

SPPU : Dec.-10, Marks 4

2. Define the following terms with example : i) Alphabet ii) String iii) Language.

SPPU : Dec.-19, End Sem, Marks 6

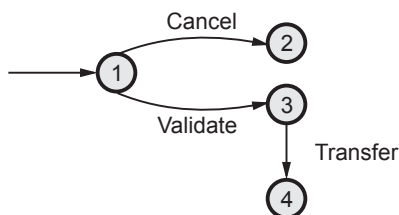
1.3 An informal Picture of FA

Finite automata is useful to solve real-world problems. To simulate any real-world problem, the events are captured by the states of finite automata. The interactions among the participants is then represented by transitions.

For example : Suppose a customer wants to purchase some books on-line. Then there are three participants involved in this scenario. Those are customer, book-store and bank. The events that would take place in such a system are -

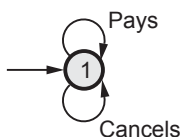
1. The customer may decide to **pay**.
The customer purchases the required book and sends the money to the store, by credit-card.
2. The customer may decide to **cancel** the purchase order. The money is sent to the bank with a message that the value of the money is to be added to the customer's bank account.
3. The book-store may **ship** books to the customer.
4. The store may get is **validated**. That means the credit card information will be validated by the bank. And it checked whether the customer's bank balance is greater than the purchase amount.

Then the finite automata for **bank** can be drawn as follows -



If customer cancels the purchase order then the money will not be transferred to the store. If customer purchases the book, then bank validates his credit card and transfers the money to the book-store.

The finite automata for **customer** can be



Customer performs only two actions either pays the money or cancel the order.

The finite automata for **book-store** can be -



The customer pays the money by credit card, the book store gets it validated from bank then the required amount of money gets transferred from customer's account to book-store's account. When the book-store gets the purchase amount then the books will be shipped to the customer.

1.4 Representation

The strings and languages can be accepted by a finite automata, when it reaches to a final state. There are two preferred notations for describing automata :

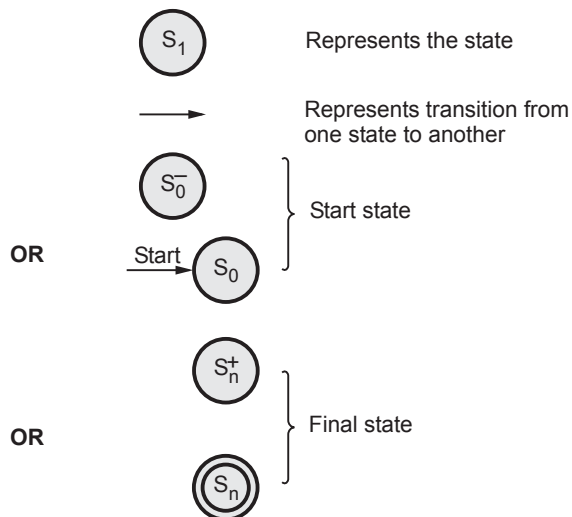
1. Transition diagram -

A transition diagram or transition graph can be defined as collection of -

- 1) Finite set of states K .
- 2) Finite set of symbols Σ .

- 3) A non empty set S of K . It is called **start state**.
- 4) A set $F \subseteq K$ of **final states**.
- 5) A transition function $K \times A \rightarrow K$ with K as state and A as input from Σ^* .

The notations used in transition diagram are -



For example :

The FA can be represented using transition graph. The machine initially is in start state S_0 then on receiving input 0 it changes to state S_1 . From S_0 on receiving input 1 the machine changes its state to S_4 . The state S_2 is a final state or accept state. When we trace the input for transition diagram and reach to a final state at end of input string then it is said that the given input is **accepted by** transition diagram.

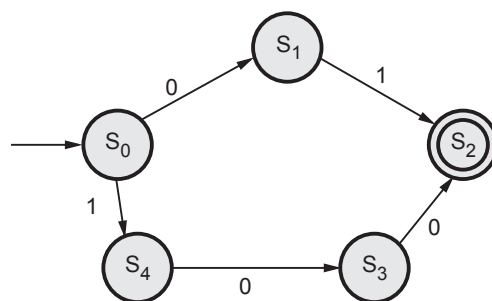


Fig. 1.4.1 Transition diagram

Another example :



We have drawn a transition diagram for the input aabb.

Note that the start state is S_0 and final state is S_4 . The input set is $\Sigma = \{a, b\}$. The states S_1 , S_2 , S_3 are all intermediate states.

2. Transition table

This is a tabular representation of finite automata. For transition table the transition function is used.

For example :

Input States	a	b
q ₀	q ₁	-
q ₁	-	q ₂
q ₂	q ₂	-

The rows of the table corresponds to states and columns of the table corresponds to inputs.

1.5 Finite State Machine (FSM)

A finite state machine is a collection of 5-tuple $(Q, \Sigma, \delta, q_0, F)$

where,

Q is a finite **set of states**, which is non empty.

Σ is **input** alphabet, indicates input set.

q_0 is an **initial state** and q_0 is in Q i.e. $q_0 \in Q$.

F is a set of **final states**.

δ is a **transition function** or a mapping function. Using this function the next state can be determined.

1.6 Language Accepted by FA

The finite automata can be represented as :

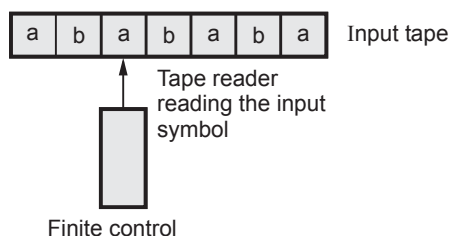


Fig. 1.6.1 Model for finite automata

The finite automata can be represented using :

- i) **Input tape** - It is a linear tape having some number of cells. Each input symbol is placed in each cell.
- ii) **Finite control** - The finite control decides the next state on receiving particular input from input tape. The tape reader reads the cells one by one from left to right and at a time only one input symbol is read.

For example, suppose current state is q , and suppose reader is reading the symbol 'a' then it is finite control which decides what will be the next state at input 'a'. The transition from current state q with input w to next state q' producing w' will be represented as,

$$(q, w) \vdash (q', w')$$

If w is a string and M is a finite automata, then w is accepted by the FA.

$$\text{iff } (w, s) \vdash^* (q, \epsilon)$$

with q as final state.

The set of strings accepted by a FA given by M then M is accepted by language L . The acceptance of M by some language L is denoted by $L(M)$.

A machine M accepts a language L iff,

$$L = L(M).$$

1.7 Definition of Regular Language

SPPU : Dec.-18, Marks 3

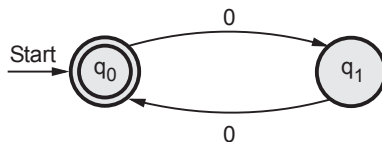
Regular languages are the sets which are accepted by finite automata.

For example : $L = \{\epsilon, 00, 0000, 000000, \dots\}$

i.e. the set of even number of zeros.

We can represent this set by a finite automata as shown in figure.

$$M = (\{q_0, q_1\}, \{0\}, \delta, q_0, q_0)$$



The given set L is said to be a regular languages because it is represented by finite automata.

Review Question

1. Define the following terms with example : i) Alphabet ii) String iii) Regular language.

SPPU : Dec.-18, In Sem, Marks 3

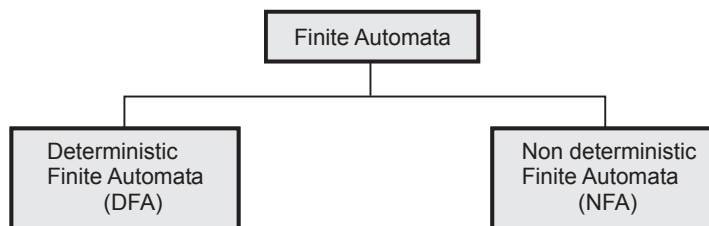
Part II : FA without Output**1.8 Types of Finite Automata**

Fig. 1.8.1 DFA and NFA

There are two types of finite automata -

- i) Deterministic Finite Automata.
- ii) Non deterministic Finite Automata.

The deterministic finite automata is deterministic in nature, in the sense, each move in this automata is uniquely determined on current input and current state. On the other hand, it is non deterministic in nature, in NFA i.e. non deterministic finite automata.

1.9 Deterministic Finite Automata

SPPU : May-10,12,13,18,19, Dec.-14,16,17,18, Aug.-15,17, Oct.-16, Marks 10

Definition of DFA

A deterministic finite automation is a collection of following things -

- 1) The finite set of states which can be denoted by Q .
- 2) The finite set of input symbols Σ .
- 3) The start state q_0 such that $q_0 \in Q$.
- 4) A set of final states F such that $F \in Q$.
- 5) The mapping function or transition function denoted by δ . Two parameters are passed to this transition function : One is current state and other is input symbol. The transition function returns a state which can be called as next state.

Example

For example, $q_0 = \delta(q_0, a)$ means from current state q_0 , with input a the next state transition is q_2 .

In short, the DFA is a five tuple notation denoted as :

$$A = (Q, \Sigma, \delta, q_0, F)$$

The name of DFA is A which is a collection of above described five elements.

It can be represented by following Fig. 1.9.1.

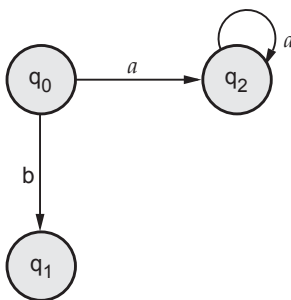


Fig. 1.9.1 Deterministic finite automata

Problems Based on DFA

Example 1.9.1 Design a FA which checks whether the given binary number is even.

Solution :

• Logic Used

The binary number is made up of 0's and 1's. When any binary number ends with 0 it is always even and when a binary number ends with 1 it is always odd. For example,

0100 is a even number, it is equal to 4.

0011 is a odd number, it is equal to 3.

• Design

While designing FA we will assume one start state, one state ending in 0 and other state for ending with 1. Since we want to check whether given binary number is even or not, we will make the state for 0, as the final state.

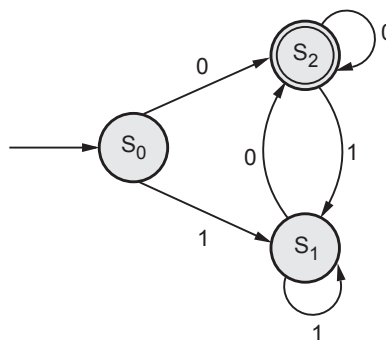


Fig. 1.9.2 DFA

• Simulation

The FA indicates clearly S_1 is a state which handles all the 1's and S_2 is a state which handles all the 0's. Let us take some input.

$$01000 \Rightarrow 0 \underline{S_2} 1000$$

$$01 \underline{S_1} 000$$

$$010 \underline{S_2} 00$$

$$0100 \underline{S_2} 0$$

$$01000 S_2$$

Now at the end of input we are in final or in accept state so it is a even number. Similarly, let us take another input.

$$1011 \Rightarrow S_0 \vdash 1011$$

$$1 S_1 \vdash 011$$

$$10 S_2 \vdash 11$$

$$101 S_1 \vdash 1$$

$$1011 S_1 \vdash$$

Now we are at the state S_1 which is a non final state. Here the symbol " $\vdash$ " denotes the symbols to be scanned. On LHS of " $\vdash$ " the already read symbols are given and at the R.H.S. of " $\vdash$ " the symbols to be scanned are given.

• Representation of DFA using Transition Table

Another idea to represent FA with the help of transition table.

States \ Input	Input	
	0	1
$\rightarrow S_0$	S_2	S_1
S_1	S_2	S_1
$\odot S_2$	S_2	S_1

Table 1.9.1 Transition table

- 1) Thus the table indicates in the first column all the current states. And under the column 0 and 1 the next states are shown.

- 2) The first row of transition table can be read as : when current state is S_0 , on input 0 the next state will be S_2 and on input 1 the next state will be S_1 . The **arrow** marked to S_0 indicates that it is a **start state** and **circle** marked to S_2 indicates that it is a **final state**. Sometimes * is also used to indicate final state.

Example 1.9.2 Design FA which accepts odd number of 1's and any number of 0's.

Solution :

• **Logic**

In the problem statement, it is indicated that there will be a state which is meant for odd number of 1's and that will be the final state. There is no condition on number of 0's.

• **Design**

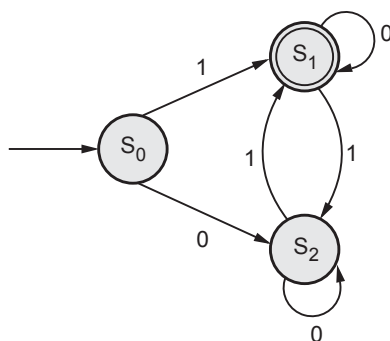


Fig. 1.9.3

Example 1.9.3 Design FA which checks whether the given unary number is divisible by 3.

Solution :

• **Logic**

The unary number is made up of ones. The number 3 can be written in unary form as 111, number 5 can be written as 11111 and so on. The unary number which is divisible by 3 can be 111 or 111111 or 111111111 and so on.

• **Design**

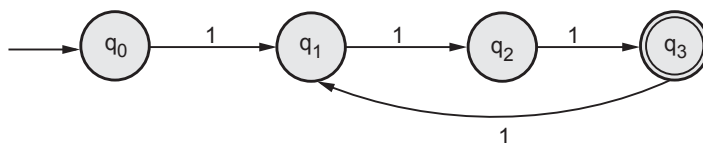


Fig. 1.9.4

Input \ State	1
$\rightarrow q_0$	q_1
q_1	q_2
q_2	q_3
q_3	q_1

• Simulation

Consider a number 111111 which is equal to 6 i.e. divisible by 3. So after complete scan of this number we reach to final state q_3 .

Start 111111

State q_0

1 q_1 1111

11 q_2 111

111 q_3 11

1111 q_1 1

11111 q_2 1

111111 q_3 $\rightarrow$ Now we are in final state.

Example 1.9.4 Design FA to check whether given decimal number is divisible by three.

Solution :

• Logic

To determine whether the given decimal number is divisible by three, we need to take the input number digit by digit. Also, while considering its divisibility by three, we have to consider that the possible remainders could be 1, 2 or 0. The remainder 0 means, it is divisible by 3. Hence from input set $\{0, 1, 2, \dots, 9\}$ (since decimal number is a input), we will get either remainder 0 or 1 or 2 while testing its divisibility by 3. So we need to group these digits according to their remainders. The groups are as given below -

remainder 0 group : S_0 : (0, 3, 6, 9)

remainder 1 group : S_1 : (1, 4, 7)

remainder 2 group : S_2 : (2, 5, 8)

• Design

We will out these states as S_0, S_1 and S_2 . The state S_0 will be the final state as it is remainder 0 state.

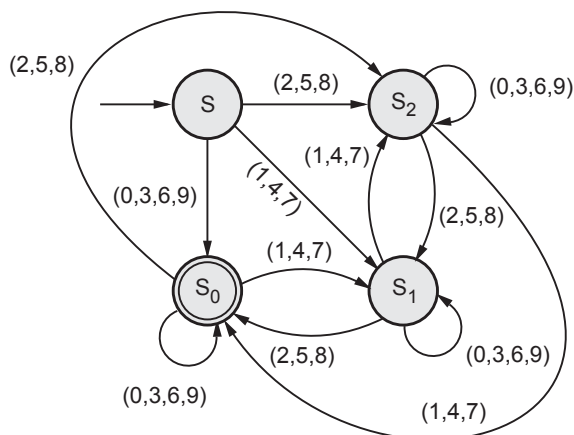


Fig. 1.9.5

• Simulation

Let us test the above FA, if the number is 36 then it will proceed by reading the number digit by digit.

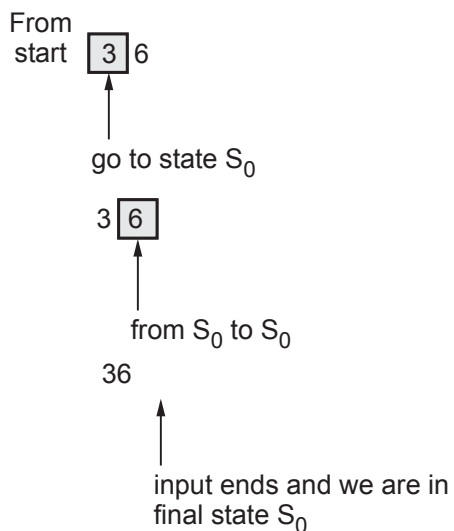


Fig. 1.9.6

Hence the number is divisible 3, isn't it ? Similarly if number is 121 which is not divisible by 3, it will be processed as

S 121

1 S₁ 21

12 S₀ 1

121 S₁ which is remainder 1 state.

Example 1.9.5 Design FA which checks whether a given binary number is divisible by three.

SPPU : May-18, End Sem, Marks 6

Solution :

• **Design**

The input number is a binary number. Hence the input set will be $\Sigma = \{0, 1\}$. The start state is denoted by S , the remainder 0 is by S_0 , remainder 1 by S_1 and remainder 2 by S_2 .

• **Simulation**

Let us take some number 010 which is equivalent to 2. We will scan this number from MSB to LSB.

0	1	0
↑	↑	↑
S_0	S_1	S_2

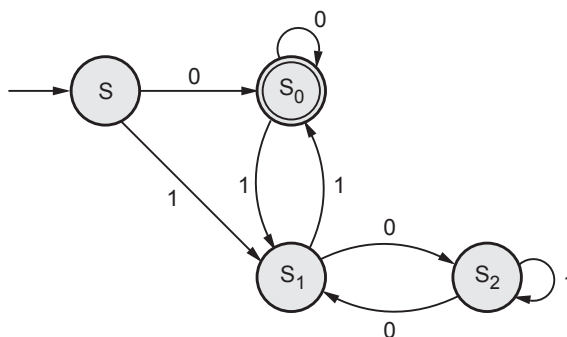


Fig. 1.9.7

Then we will reach to state S_2 which is remainder 2 state. Similarly, for input 1001 which is equivalent to 9 we will be in final state after scanning the complete input.

1	0	0	1
↑	↑	↑	↑
S_1	S_2	S_1	S_0

Thus the number is really divisible by 3.

Example 1.9.6 Design FA which accepts even number of 0's and even number of 1's.

Solution :

• **Logic**

This FA will consider four different stages for input 0 and 1. The stages could be :

- even number of 0 and even number of 1,
- even number of 0 and odd number of 1,
- odd number of 0 and even number of 1,
- odd number of 0 and odd number of 1.

• Design

Let us try to design the machine :

Here q_0 is a start state as well as final state. Note carefully that a symmetry of 0's and 1's is maintained. We can associate meanings to each state as :

q_0 : State of even number of 0's and even number of 1's.

q_1 : State of odd number of 0's and even number of 1's.

q_2 : State of odd number of 0's and odd number of 1's.

q_3 : State of even number of 0's and odd number of 1's.

The transition table can be as follows -

	0	1
→ q_0	q_1	q_3
q_1	q_0	q_2
q_2	q_3	q_1
q_3	q_2	q_0

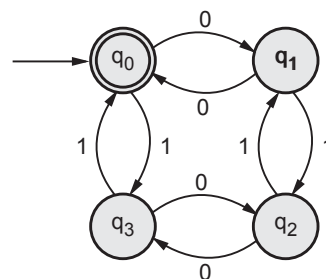


Fig. 1.9.8

Example 1.9.7 Design FA to accept the string that always ends with 00.

Solution :

If the input is 01001100 it will be processed as :

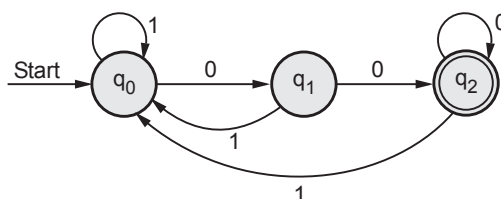


Fig. 1.9.9

The q_2 is a final state, hence the input is accepted.

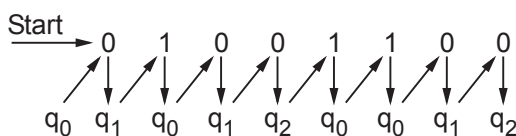


Fig. 1.9.10

Example 1.9.8 Construct the transition graph for a FA which accepts a language L over $\Sigma \{0, 1\}$ in which every string starts with 0 and ends with 1.

Solution :

In the above transition graph we have handled the case as if the input starts with 1 then it will be in q_3 state which is a dead state and never lead to final state. Thus this machine strictly handles the strings starting with 0 and ending with 1.

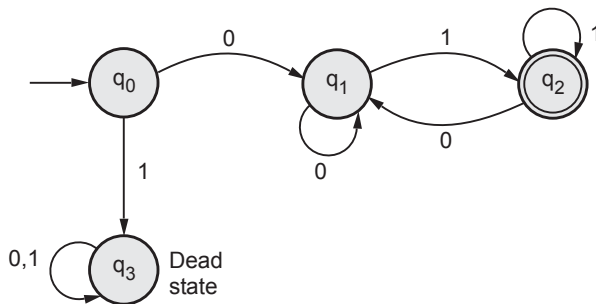


Fig. 1.9.11

Example 1.9.9 Design FA to accept L , where $L = \{\text{Strings in which } a \text{ always appears trippled}\}$ over the set $\Sigma = \{a, b\}$.

SPPU : Aug.-17, In Sem, Marks 3

Solution :

• **Logic**

For this particular language the valid strings are $aaab$, $baaaaa$, $bbaaab$ and so on. The a always appears in a clump of 3. The TG (transition graph) will look like this -

• **Design**

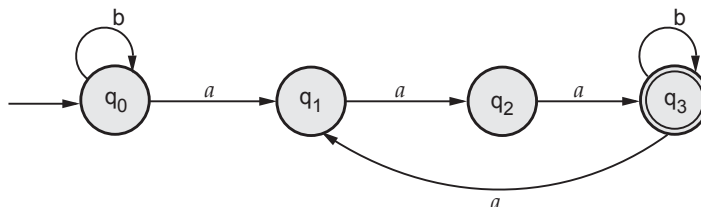


Fig. 1.9.12

Note that the sequence of tripple a is maintained to reach to the final state.

Example 1.9.10 Design FA to accept L where all the strings in L are such that total number of a 's in them are divisible by 3.

Solution :

• **Logic**

As we have seen earlier, while testing divisibility by 3, we group the input as remainder 0, remainder 1 and remainder 2.

Hence,

S_0 : State of remainder 0.

S_1 : State of remainder 1.

S_2 : State of remainder 2.

• Design

Note the difference between previous example and this, here there is no condition as a should be in chunk but total number of a 's in a string are divisible by 3. Hence b 's are allowed in between.

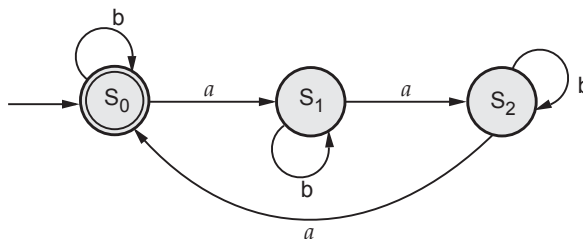


Fig. 1.9.13

Example 1.9.11 Design a FA that reads strings made up of letters in the word. CHARIOT and recognize those strings that contain the word 'CAT' as a substring.

Solution :

• Logic

To design this FA we will first consider the input set which will be $\Sigma = \{C, H, A, R, I, O, T\}$. From this input set we have to recognize the word 'CAT'. We can do that as follows -

Step 1 :



Fig. 1.9.14

Step 2 :

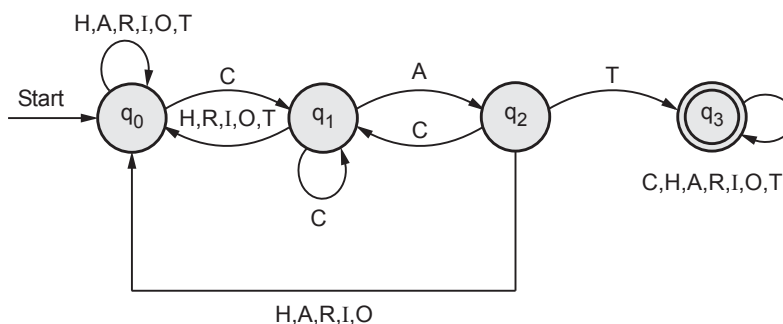


Fig. 1.9.15 Required finite automata

From Fig. 1.9.15 it is clear that on recognizing C initially from $\Sigma = \{C, H, A, R, I, O, T\}$ we are moving to state q_1 , other than C all the characters are remaining in state q_0 only. From q_1 the character A will be identified and from q_2 character T will be identifier. The transition table can be drawn as follows :

• Design

State \ Input	C	H	A	R	I	O	T
$\rightarrow q_0$	q_1	q_0	q_0	q_0	q_0	q_0	q_0
q_1	q_1	q_0	q_2	q_0	q_0	q_0	q_0
q_2	q_1	q_0	q_0	q_0	q_0	q_0	q_3
$\odot q_3$	q_3	q_3	q_3	q_3	q_3	q_3	q_3

The substring may appear anywhere in the input string for any number of times. For example, 'HACCATHA'. From this 'CAT' can be recognized and we should reach to q_3 state finally.

Example 1.9.12 Design DFA to accept odd and even numbers represented using binary notation.

Solution :

• Logic

The binary number that ends with 0 is an even number and binary number that ends with 1 is an odd number. Hence the DFA is -

• Design

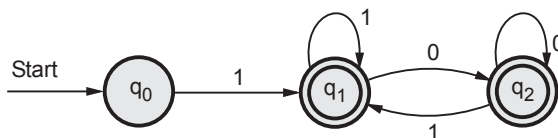


Fig. 1.9.16

Example 1.9.13 Write a DFA to accept the language $L = \{L : |W| \bmod 5 \neq 0\}$.

Solution :

• Logic

The string which we obtain should not be divisible by 5. Hence the, DFA will be,

• Design

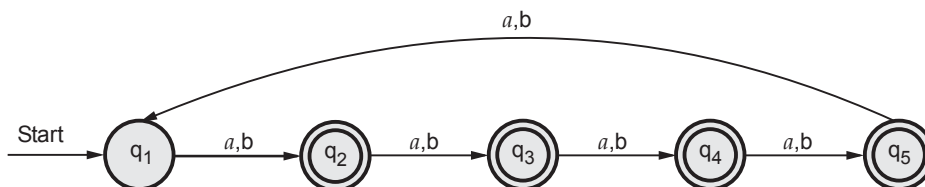


Fig. 1.9.17

The transition table can be drawn as follows -

State \ Input	a	b
q ₁	q ₂	q ₂
q ₂	q ₃	q ₃
q ₃	q ₄	q ₄
q ₄	q ₅	q ₅
q ₅	q ₁	q ₁

We can consider the string "abbb". This string is a valid string as it is not divisible by 5 i.e. $abbb \bmod 5 \neq 0$. That also means that we should reach to final state on acceptance of this string. The simulation of this string for given DFA is :

• Simulation

$$\begin{aligned}
 \delta(q_1, abbb) &\vdash \delta(q_2, bbb) \\
 &\vdash \delta(q_3, bb) \\
 &\vdash \delta(q_4, b) \\
 &\vdash \delta(q_5, \epsilon)
 \end{aligned}$$

where q_5 is a final state.

Now consider string "ababa" which is invalid string as it is divisible by 5. That means $ababa \bmod 5 = 0$. That means we should reach to non final state on acceptance of this string. The simulation for this string is as given below.

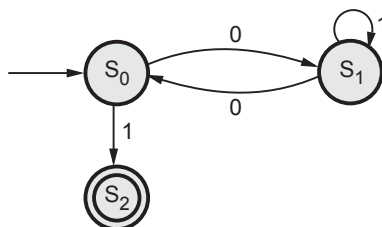
As q_1 is a start we will start from state q_1 .

$$\begin{aligned}
 \delta(q_1, ababa) &\vdash \delta(q_2, baba) \\
 &\vdash \delta(q_3, aba) \\
 &\vdash \delta(q_4, ba) \\
 &\vdash \delta(q_5, a) \\
 &\vdash \delta(q_1, \epsilon)
 \end{aligned}$$

Here q_1 is a non-final state. That means "ababa" is invalid string for the given DFA.

Example 1.9.14 Design a DFA which accepts strings with even number of 0's followed by single 1 over $\Sigma = \{0, 1\}$.

Solution : The DFA can be shown by a transition diagram as :



Here state S_1 will accept all the odd number of 0's and S_0 will accept all even number of 0's. It should then be followed by single 1. Hence S_2 is a final state.

- Consider the input :

```

0 0 0 1 0 1
0 S1 0 0 1 0 1
0 0 S0 0 1 0 1
0 0 0 S1 1 0 1
0 0 0 1 S1 0 1
0 0 0 1 0 S0 1
0 0 0 1 0 1 S2
  
```

We now will reach to final state S_2 and the input ends here. Hence 000101 is a valid input string. The transition table for above drawn DFA can be represented as

States \ Input	Input	
	0	1
→ S_0	S_1	S_2
S_1	S_0	S_1
S_2	-	-

Table 1.9.2 Transition table

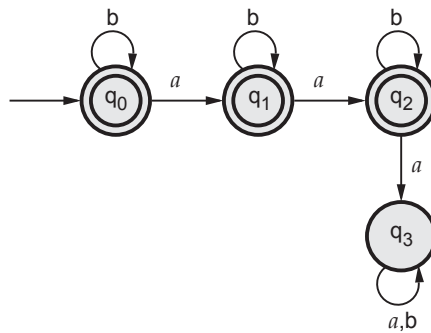
Example 1.9.15 Design DFA which accepts all the strings not having more than two a's over $\Sigma = \{a, b\}$.

Solution :

- Logic**

In this DFA maximum two a's are accepted. If we try to accept third a then it should not lead us to final state.

• Design



The transition table can be as shown -

States \ Input	Input	
	a	b
→ q ₀	q ₁	q ₀
q ₁	q ₂	q ₁
q ₂	q ₃	q ₂
q ₃	q ₃	q ₃

Table 1.9.3 Transition table

Example 1.9.16 Design a DFA for accepting all the strings of $\{L = 0^m 1^n \mid m \geq 0 \text{ and } n \geq 1\}$.

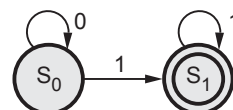
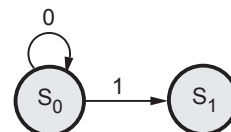
Solution :

• Logic

This is a language in which all the 0's followed by all 1's. But number of zero's and number of 1's are different. The language explicitly mentions that there should be at least one 1. Any number of 0's followed by only one 1 is -

• Design

We have 1^n in the language. Hence the DFA can be



The transition table can be

State \ Input	0	1
S_0	S_0	S_1
S_1	–	S_1

• Simulation

Consider a string 00111, which is a valid string and is accepted by above drawn DFA.

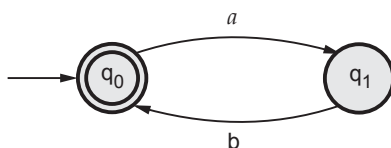
$$\begin{aligned}
 \delta(S_0, 00111) &\vdash (S_0, 0111) \\
 &\vdash (S_0, 111) \\
 &\vdash (S_1, 11) \\
 &\vdash (S_1, 1) \\
 &\vdash (S_1, \epsilon)
 \end{aligned}$$

Thus we reach to final state S_1 on acceptance of 00111.

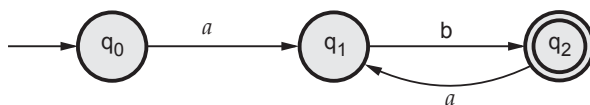
Example 1.9.17 Design DFA over $\Sigma = \{a, b\}$ for i) $(ab)^n$ with $n \geq 0$ ii) $(ab)^n$ with $n \geq 1$.

Solution : This is a language of ab in a pair in which i) Condition ϵ is accepted and in ii) Condition ϵ is not accepted.

The DFA for i) can be



ii) The DFA will be -



The string accepted by

$$\begin{aligned}
 \text{i) } \delta(q_0, ab) &\vdash (q_1, b) \\
 &\vdash (q_0, \epsilon)
 \end{aligned}$$

Here q_0 is a final state.

$$\begin{aligned} \text{ii) } \delta(q_0, ab) &\vdash (q_1, b) \\ &\vdash (q_2, \epsilon) \end{aligned}$$

Here q_2 is final state.

Example 1.9.18 Design DFA for accepting the set of integers.

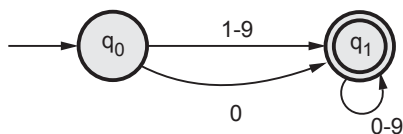
Solution :

• **Logic**

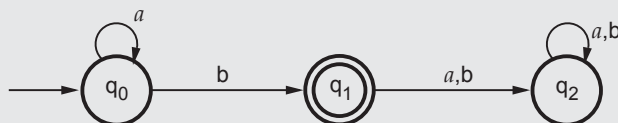
For representing any integer the set $\{0, 1, \dots, 9\}$ is used. We can represent any integer value by taking appropriate combinations of symbols from $\{0, \dots, 9\}$.

• **Design**

DFA will be



Example 1.9.19 Recognize the language given by following DFA.



Solution : In the given DFA, at q_0 state a self loop for symbol a is given. That means any number of a 's are allowed. Then if single b comes then it leads to final state. But after this b if anything i.e. a or b comes then we reach to a non final state q_2 and in this state whatever may come, we will be in q_2 state only. Basically state q_2 is called a dead state. Thus it is a language in which any number of a 's should be followed by single b only. Hence this language is denoted as

$$\{L = a^n b \mid n \geq 0\}.$$

Example 1.9.20 Design DFA for a language of string 0 and 1 that

I) Ending with 10 II) Ending with 11

III) Ending with 1.

SPPU : May-10, Marks 10

Solution :

I) **DFA which ends with 10**

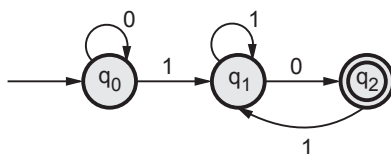


Fig. 1.9.18

II) DFA which ends with 11

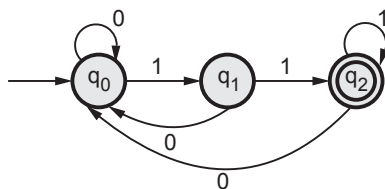


Fig. 1.9.19

III) DFA which ends with 1

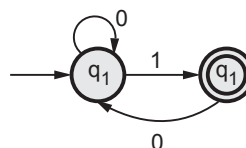


Fig. 1.9.20

Example 1.9.21 Construct DFA for checking "whether a string over alphabet $\{a, b\}$ contains a substring abb ".

SPPU : May-12, Marks 8

Solution : The DFA will be -

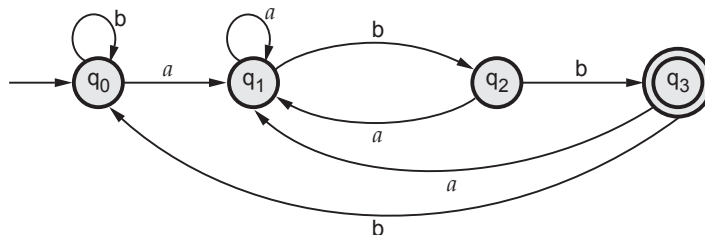


Fig. 1.9.21

Example 1.9.22 Design a finite automata that reads strings made up of letters in the word 'CHARIOT' and recognize those strings that contain the word 'CAT' as a substring.

SPPU : May-13, Marks 8

Solution :

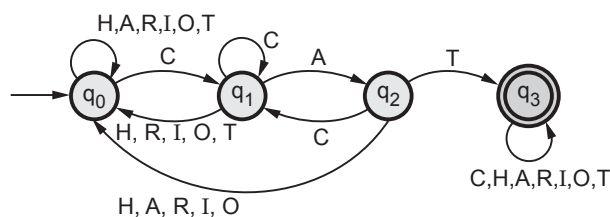


Fig. 1.9.22

Input \ State	C	H	A	R	I	O	T
→ q ₀	q ₁	q ₀	q ₀	q ₀	q ₀	q ₀	q ₀
q ₁	q ₁	q ₀	q ₂	q ₀	q ₀	q ₀	q ₀
q ₂	q ₁	q ₀	q ₀	q ₀	q ₀	q ₀	q ₃
q ₃	q ₃	q ₃	q ₃	q ₃	q ₃	q ₃	q ₃

$$M = (\{q_0, q_1, q_2, q_3\}, \{C, H, A, R, I, O, T\}, \delta, q_0, \{q_3\})$$

Example 1.9.23 Design FSM over $\{a, b\}$ accepting the string where number of a 's are divisible by 2 and number of b 's are divisible by 3. **SPPU : Dec.-14, End Sem, Marks 6**

Solution :

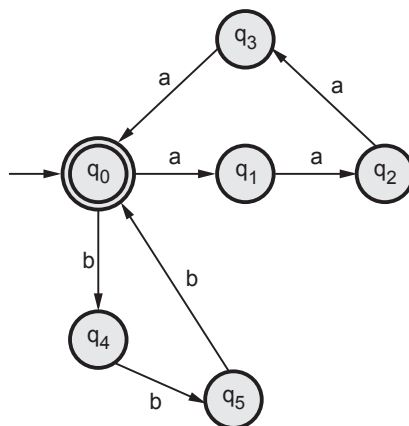
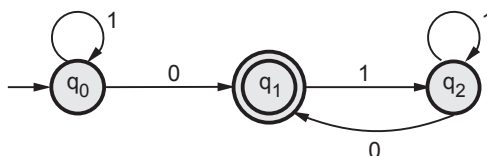


Fig. 1.9.23

Example 1.9.24 Draw FA for strings which do not contain '00' as substring in it over alphabet $\Sigma = \{0, 1\}^*$ **SPPU : Aug.-15, In Sem, Marks 2**

Solution :



Example 1.9.25 Construct a DFA over the alphabets $\{0, 1\}$ for accepting the strings having number of 1's as multiple of 3. **SPPU : Dec.-16, End Sem, Marks 6**

Solution :

- **Logic**

Here the condition is on number of 1's and not on number of 0's.

- **Design**

Hence DFA is as follows :

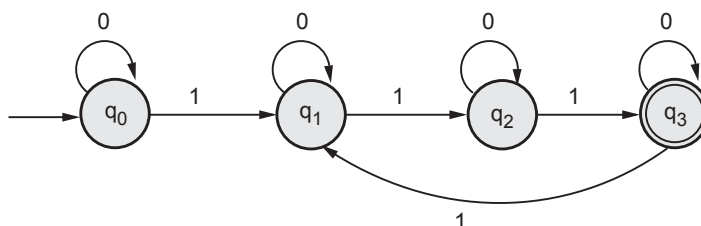


Fig. 1.9.24

• **Simulation :**

Consider string 0100110

$q_0 \vdash 0100110$

$0q_0 \vdash 100110$

$01q_1 \vdash 00110$

$010q_1 \vdash 0110$

$0100q_0 \vdash 110$

$01001q_2 \vdash 10$

$010011q_3 \vdash 0$

$0100110q_3$

ACCEPT

Example 1.9.26 Design Finite Automata (FA) for accepting strings, over $\Sigma = \{0, 1\}$, with even number of 0's and odd number of 1's. **SPPU : Oct.-16, In Sem, Marks 4**

Solution : The finite automata will be as follows -

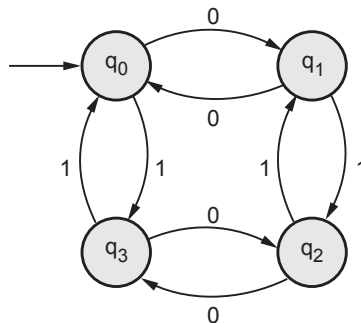


Fig. 1.9.25

Example 1.9.27 Construct the DFA for the language of all strings that begin and end with same symbol over the alphabet $\Sigma = \{0, 1\}$. **SPPU : Aug.-17, In Sem, Marks 4**

Solution :

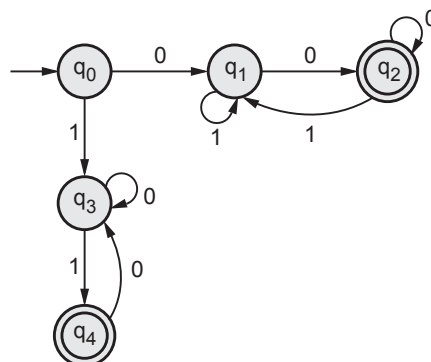


Fig. 1.9.26

Example 1.9.28 Construct DFA for language defined by $\Sigma = \{0, 1\}$ where

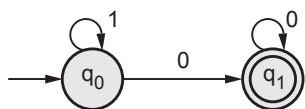
$S = \{\text{Strings ending with 0 always}\}$

$S = \{\text{strings representing odd binary numbers}\}$

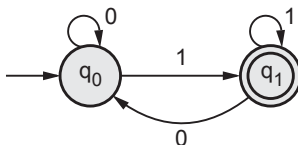
$S = \{\text{strings over } \Sigma^* \text{ with total number of 0's even}\}$

SPPU : Dec.-17, End Sem, Marks 6

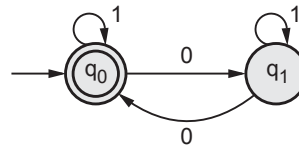
Solution : i)



ii)



iii)



Example 1.9.29 Design a FA which checks the divisibility by 4 for a decimal number.

SPPU : Dec.-17, End Sem, Marks 6

Solution : For checking divisibility by 4 for a decimal number we have to consider remainder 0, 1, 2 and 3 groups from input set $\Sigma = \{0, 1, 2, 3, 4, 5, 6, 7, 8\}$. We will name these groups as q_0 , q_1 , q_2 and q_3 states.

$$q_0 = \{0, 4, 8\}$$

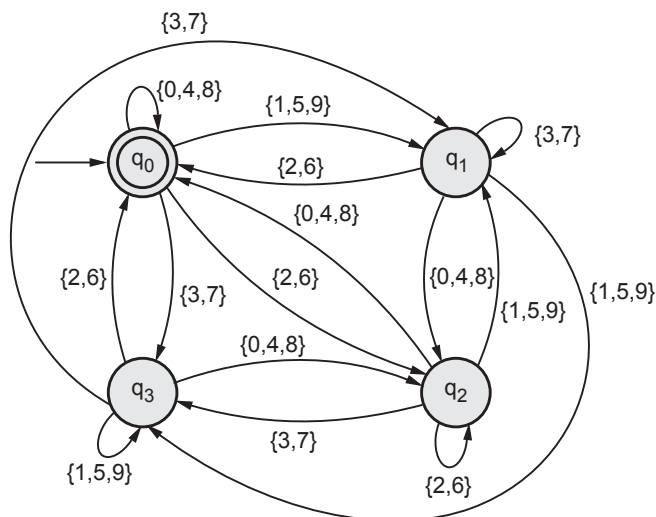
$$q_1 = \{1, 5, 9\}$$

$$q_2 = \{2, 6\}$$

$$q_3 = \{3, 7\}$$

Here q_0 is a start state as well as final state. The transition table as well as transition diagram is as given below.

State \ i/p	{0, 4, 8}	{1, 5, 9}	{2, 6}	{3, 7}
q_0	q_0	q_1	q_2	q_3
q_1	q_2	q_3	q_0	q_1
q_2	q_0	q_1	q_2	q_3
q_3	q_2	q_3	q_0	q_1



Example 1.9.30 Explain the application of regular expressions in text search and replace.

SPPU : May-18, End Sem, Marks 6

Solution : Regular expressions are useful notations used for finding the patterns from given text. A large class of pattern can be identified by regular expression.

For example : The strings ending with digits is a pattern in the given text which can be recognized by following regular expression -

$[a-zA-Z]^+ [0-9]^+$

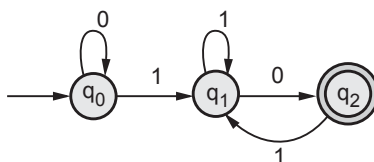
Thus use of regular expressions for identifying patterns from given text makes the job of compiler more simplified.

In web applications also the pattern matching is done using regular expressions.

Example 1.9.31 Construct a DFA to accept strings ending with 10.

SPPU : Oct.-18, In Sem, Marks 4

Solution :



Example 1.9.32 Design a DFA which accepts a ternary number divisible by 4.

SPPU : Dec.-18, End Sem, Marks 6

Solution : The ternary number is denoted by $\Sigma = \{0, 1, 2\}$.

For divisibility by 4 there are four cases -

Remainder 0, Remainder 1, Remainder 2 and Remainder 3.

Now we will consider separate states for each of these cases.

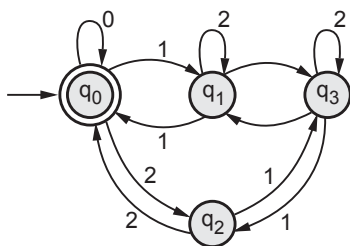
q_0 - Remainder 0 state

q_1 - Remainder 1 state

q_2 - Remainder 2 state

q_3 - Remainder 3 state

Here q_0 is start state as well as final state. The FA is,



For example

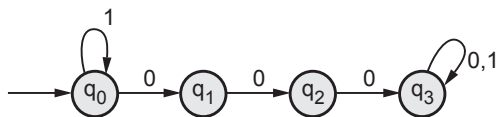
$(121)_3 = (16)_{10}$ which is divisible by 4.

Example 1.9.33 Design FA accepting the following language over the alphabet $\{0, 1\}$

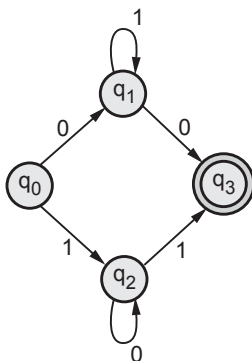
- Set of all strings having at least three consecutive zeros.
- Set of all strings that begin and end with same symbol.

SPPU : Dec.-18, End Sem, Marks 8

Solution : i)



ii)

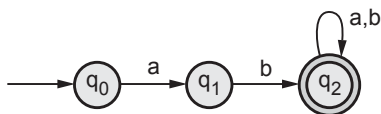


Example 1.9.34 Construct a DFA accepting the following languages over the alphabet $\{a, b\}$

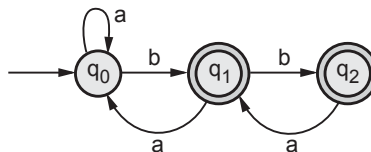
- Set of all strings that begin with the substring ab .
- Set of all strings with at most two consecutive b 's.

SPPU : Dec.-18, End Sem, Marks 8

Solution : i)



ii)



Example 1.9.35 Construct FA for the following language L .

$L = \{w/w \text{ is a binary word of length } 4i, i \geq 1 \text{ such that each consecutive block 4 bits contains at least 2 0's}\}$

SPPU : May-19, End Sem, Marks 8

Solution :

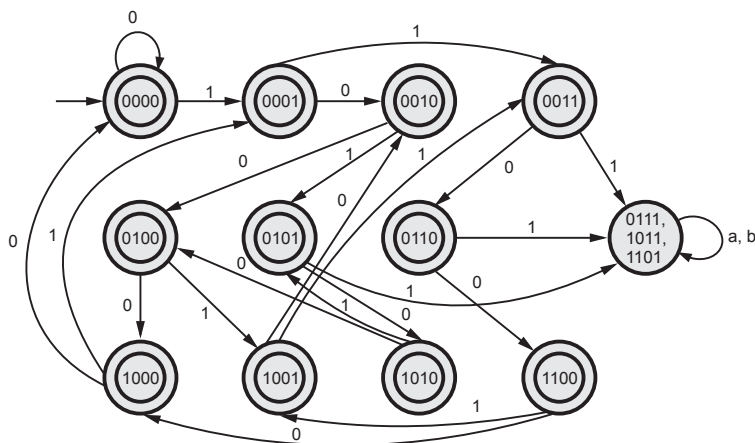


Fig. 1.9.27

Review Question

1. Give formal definition of - Deterministic Finite Automata.

SPPU : May-19, End Sem, Marks 2

1.10 Non Deterministic Finite Automata

SPPU : Dec.-12,13,14, May-12,14, Aug.-15,17, Marks 6

Definition of NFA

The NFA can be formally defined as a collection of 5-tuples.

- 1) Q is a finite set of states.
- 2) Σ is a finite set of inputs.
- 3) δ is called next state or transition function.
- 4) q_0 is initial state.
- 5) F is a final state where $F \subseteq Q$.

There can be multiple final states. Thus the next question might be what is the use of NFA. The NFA is basically used in theory of computations because they are more flexible and easier to use than the DFAs.

Example

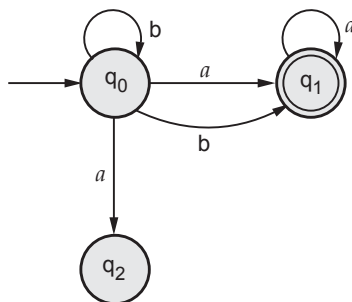


Fig. 1.10.1 Non deterministic finite automata

Difference in NFA and DFA

Sr. No.	NFA	DFA
1.	NFA stands for Non deterministic Finite Automata.	DFA stands for Deterministic Finite Automata
2.	For every input symbol there can be more than one state transition.	For every input symbol there can be only one state transition.
3.	NFA can use empty string transition.	DFA can not use empty string transition.
4.	Construction of NFA is simple	Construction of DFA is difficult

Problems Based on NFA

Example 1.10.1 An NFA with states 1-5 and input alphabet $\{a, b\}$ has following transition table :

q	$\delta(q, a)$	$\delta(q, b)$
1	{1, 2}	{1}
2	{3}	{3}
3	{4}	{4}
4	{5}	ϕ
5	ϕ	{5}

i) Draw transition diagram ii) Calculate $\delta^*(1, ab)$ iii) Calculate $\delta^*(1, abaab)$

SPPU : May-14, Marks 6

Solution :

i) Transition diagram

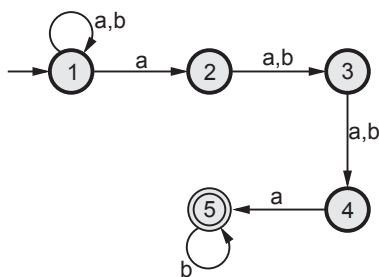


Fig. 1.10.2

ii) $\delta^*(1, ab) = 1 \text{ or } 3$

iii) $\delta^*(1, abaab) = 1 \text{ or } 3 \text{ or } 4 \text{ or } 5$

Example 1.10.2 Give non-deterministic finite automata to accept the following language over $\{0, 1\}^*$. The set of all strings containing either 101 or 110 as a substring.

SPPU : Dec.-13, Marks 4

Solution :

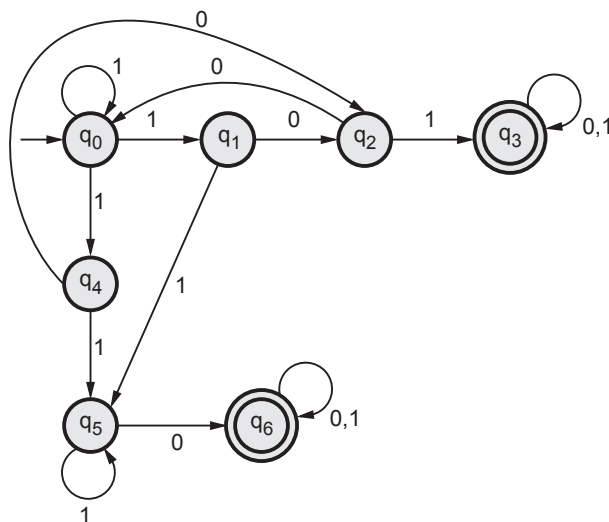


Fig. 1.10.3

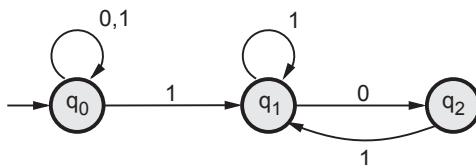
Example 1.10.3 Is NFA more powerful than DFA ? Justify **SPPU : Dec.-14, In Sem, Marks 2**

Solution : The NFA is not more powerful than DFA. In fact NFA can be converted to equivalent DFA. Thus any regular language can be represented by both NFA and DFA.

Example 1.10.4 Draw NFA for strings ending with '10' over alphabet $\Sigma = \{0,1\}^*$.

SPPU : Aug.-15, In Sem, Marks 2

Solution :



Review Questions

1. Differentiate between NFA and DFA.

SPPU : May-12, Marks 2, May-14, Marks 4, Aug-17, In Sem, Marks 3

2. Define formally with example - Non deterministic finite automata.

SPPU : Dec.-12, May-14, Marks 2

1.11 Epsilon- NFA

SPPU : Dec.-13,18,19, Aug.-17, May-11, Marks 5

The ϵ -transitions in NFA are given in order to move from one state to another without having any symbol from input set Σ .

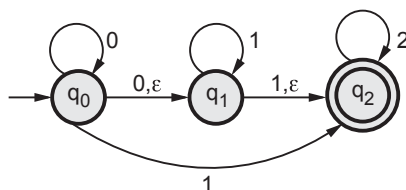


Fig. 1.11.1

- In this NFA with ϵ , q_0 is a **start state** with input 0.
- Initially with **input 0**, we can go to q_0 or q_1 .
- Similarly with **input 1**, we can go to state q_1 or q_2 .
- Thus it is not definite that on input 1 whether we will be in state q_1 or q_2 . Hence it is called **non-deterministic Finite Automata (NFA)** and since there are some ϵ moves to simply change the states, it is called **NFA with ϵ** .

Definition

The language L accepted by NFA with ϵ , denoted by $M = (Q, \Sigma, \delta, q_0, F)$ can be defined as :

Let, $M = (Q, \Sigma, \delta, q_0, F)$ be a NFA with ϵ .

where

Q is a finite set of states.

Σ is input set.

δ is a transition or a mapping function for transitions from $Q \times \{\Sigma \cup \epsilon\}$ to 2^Q .

q_0 is a start state.

F is a set of final states such that $F \subseteq Q$.

The string w in L accepted by NFA can be represented as

$L(M) = \{w \mid w \in \Sigma^* \text{ and } \delta \text{ transition for } w \text{ from } q_0 \text{ reaches to } F\}$.

Example 1.11.1 Construct NFA with ϵ which accepts a language consisting the strings of any number of a 's followed by any number of b 's. Followed by any number of c 's.

SPPU : Dec.-19, End Sem, Marks 3

Solution : Here any number of a 's or b 's or c 's means zero or more in number. That means there can be zero or more a 's followed by zero or more b 's followed by zero or more c 's. Hence NFA with ϵ can be -

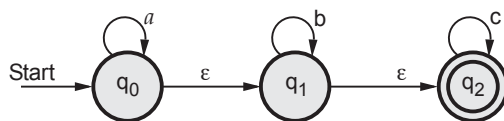


Fig. 1.11.2

Normally ϵ 's are not shown in the input string. The transition table can be -

δ		Σ			
State	Input	a	b	c	ϵ
q_0		q_0	ϕ	ϕ	q_1
q_1		ϕ	q_1	ϕ	q_2
q_2		ϕ	ϕ	q_2	ϕ

We can parse the string *aabbcc* as follows -

$$\begin{aligned}
 \delta(q_0, aabbcc) &\vdash \delta(q_0, abbcc) \\
 &\vdash \delta(q_0, bbcc) \\
 &\vdash \delta(q_0, \epsilon bcc) \\
 &\vdash \delta(q_1, bbcc) \\
 &\vdash \delta(q_1, bcc) \\
 &\vdash \delta(q_1, cc) \\
 &\vdash \delta(q_1, \epsilon cc) \\
 &\vdash \delta(q_2, cc) \\
 &\vdash \delta(q_2, c) \\
 &\vdash \delta(q_2, \epsilon)
 \end{aligned}$$

Thus we reach to accept state, after scanning the complete input string.

Example 1.11.2 Compare NFA and NFA ϵ .

SPPU : Aug.-17, In Sem, Marks 3

Solution :

- The NFA and NFA ϵ both are non-deterministic in nature.
- There are no ϵ (empty) transitions in NFA whereas there are empty transitions present in NFA ϵ .
- NFA occupies less space whereas NFA ϵ occupies more space.

1.11.1 ϵ - Closure

The ϵ - closure (p) is a set of all states which are reachable from state p on ϵ -transitions such that :

- ϵ - closure (p) = p where $p \in Q$.
- If there exists ϵ - closure (p) = $\{q\}$ and $\delta(q, \epsilon) = r$ then ϵ - closure (p) = $\{q, r\}$.

Example 1.11.3 Find ϵ - closure for the following NFA with ϵ .

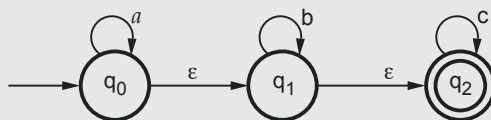


Fig. 1.11.3

Solution :

ϵ - closure $(q_0) = \{q_0, q_1, q_2\}$ means self state + ϵ - reachable states.

ϵ - closure $(q_1) = \{q_1, q_2\}$ means q_1 is a self state and q_2 is a state obtained from q_1 with ϵ input.

ϵ - closure $(q_2) = \{q_2\}$

Review Questions

1. Describe - NFA with null moves. **SPPU : May-11, Marks 2**
2. Give the formal definitions of following with suitable examples : Epsilon closure, non-deterministic finite automata and deterministic finite automata. **SPPU : Dec.-13, Marks 5**
3. Explain the extended transition function NFA with epsilon. **SPPU : Dec.-18, In Sem, Marks 3**

1.12 NFA with ϵ to NFA without ϵ **SPPU : Dec.-14,15, Oct.-16, May-19, Marks 6**

Steps for Conversion

1. Find out all the ϵ transitions from each state from Q. That will be called as ϵ - closure $\{q_i\}$ where $q_i \in Q$.
2. Then δ' transitions can be obtained. The δ' transitions means an ϵ -closure on δ moves.
3. Step - 2 is repeated for each input symbol and for each state of given NFA.
4. Using the resultant states the transition table for equivalent NFA without ϵ can be built.

$$\delta'(q, a) = \epsilon - \text{closure}(\delta(\hat{\delta}(q, \epsilon), a))$$

where

$$\hat{\delta}(q, \epsilon) = \epsilon - \text{closure}(q)$$

Example 1.12.1 Convert the given NFA with ϵ to NFA without ϵ .

SPPU : Dec.-14, Oct.-16, In Sem, Marks 4, Dec.-15, End Sem, Marks 6

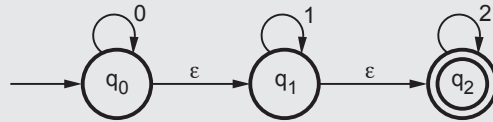


Fig. 1.12.1

Solution : We will first obtain ϵ - closure of each state i.e. we will find out ϵ - reachable states from current state.

Hence

$$\epsilon - \text{closure}(q_0) = \{q_0, q_1, q_2\}$$

$$\epsilon - \text{closure}(q_1) = \{q_1, q_2\}$$

$$\epsilon - \text{closure}(q_2) = \{q_2\}$$

As ϵ - closure (q_0) means with null input (no input symbol) we can reach to q_0 , q_1 or q_2 . In a similar manner for q_1 and q_2 ϵ - closures are obtained. Now we will obtain δ' transitions for each state on each input symbol.

$$\begin{aligned} \delta'(q_0, 0) &= \epsilon - \text{closure} \left(\delta(\hat{\delta}(q_0, \epsilon), 0) \right) \\ &= \epsilon - \text{closure} \left(\delta(\epsilon - \text{closure}(q_0), 0) \right) \\ &= \epsilon - \text{closure} \left(\delta(q_0, q_1, q_2), 0 \right) \\ &= \epsilon - \text{closure} \left(\delta(q_0, 0) \cup \delta(q_1, 0) \cup \delta(q_2, 0) \right) \\ &= \epsilon - \text{closure} (q_0 \cup \phi \cup \phi) \\ &= \epsilon - \text{closure} (q_0) = \{q_0, q_1, q_2\} \end{aligned}$$

$$\begin{aligned} \delta'(q_0, 1) &= \epsilon - \text{closure} \left(\delta(\hat{\delta}(q_0, \epsilon), 1) \right) \\ &= \epsilon - \text{closure} \left(\delta(q_0, q_1, q_2), 1 \right) \\ &= \epsilon - \text{closure} \left(\delta(q_0, 1) \cup \delta(q_1, 1) \cup \delta(q_2, 1) \right) \\ &= \epsilon - \text{closure} (\phi \cup q_1 \cup \phi) \\ &= \epsilon - \text{closure} (q_1) \end{aligned}$$

$$\delta'(q_0, 1) = \{q_1, q_2\}$$

$$\begin{aligned} \delta'(q_1, 0) &= \epsilon - \text{closure} \left(\delta(\hat{\delta}(q_1, \epsilon), 0) \right) \\ &= \epsilon - \text{closure} \left(\delta(\epsilon - \text{closure}(q_1), 0) \right) \\ &= \epsilon - \text{closure} \left(\delta(q_1, q_2), 0 \right) \end{aligned}$$

$$= \varepsilon - \text{closure} (\delta(q_1, 0) \cup \delta(q_2, 0))$$

$$= \varepsilon - \text{closure} (\phi \cup \phi)$$

$$= \varepsilon - \text{closure} (\phi)$$

$$= \phi$$

$$\delta'(q_1, 1) = \varepsilon - \text{closure} (\delta(\hat{\delta}(q_1, \varepsilon), 1))$$

$$= \varepsilon - \text{closure} (\delta(\varepsilon - \text{closure}(q_1), 1))$$

$$= \varepsilon - \text{closure} (\delta(q_1, q_2), 1)$$

$$= \varepsilon - \text{closure} (\delta(q_1, 1) \cup \delta(q_2, 1))$$

$$= \varepsilon - \text{closure} (q_1 \cup \phi)$$

$$= \varepsilon - \text{closure} (q_1)$$

$$= \{q_1, q_2\}$$

$$\delta'(q_2, 0) = \varepsilon - \text{closure} (\delta(\hat{\delta}(q_2, \varepsilon), 0))$$

$$= \varepsilon - \text{closure} (\delta(\varepsilon - \text{closure}(q_2), 0))$$

$$= \varepsilon - \text{closure} (\delta(q_2, 0))$$

$$= \varepsilon - \text{closure} (\phi)$$

$$= \phi$$

$$\delta'(q_2, 1) = \varepsilon - \text{closure} (\delta(\hat{\delta}(q_2, \varepsilon), 1))$$

$$= \varepsilon - \text{closure} (\delta(\varepsilon - \text{closure}(q_2), 1))$$

$$= \varepsilon - \text{closure} (\delta(q_2, 1))$$

$$= \varepsilon - \text{closure} (\phi)$$

$$\delta'(q_2, 1) = \phi$$

$$\delta'(q_0, 2) = \varepsilon - \text{closure} (\delta(\hat{\delta}(q_0, \varepsilon), 2))$$

$$= \varepsilon - \text{closure} (\delta(\varepsilon - \text{closure}(q_0), 2))$$

$$= \varepsilon - \text{closure} (\delta(q_0, q_1, q_2), 2)$$

$$= \varepsilon - \text{closure} (\delta(q_0, 2) \cup \delta(q_1, 2) \cup \delta(q_2, 2))$$

$$= \varepsilon - \text{closure} (\phi \cup \phi \cup q_2)$$

$$= \varepsilon - \text{closure} (q_2)$$

$$= \{q_2\}$$

$$\begin{aligned}
\delta'(q_1, 2) &= \varepsilon\text{-closure} \left(\delta \left(\hat{\delta}(q_1, \varepsilon), 2 \right) \right) \\
&= \varepsilon\text{-closure} \left(\delta(\varepsilon\text{-closure}(q_1), 2) \right) \\
&= \varepsilon\text{-closure} \left(\delta(q_1, q_2), 2 \right) \\
&= \varepsilon\text{-closure} \left(\delta(q_1, 2) \cup \delta(q_2, 2) \right) \\
&= \varepsilon\text{-closure} (\phi \cup q_2) \\
&= \{q_2\} \\
\delta'(q_2, 2) &= \varepsilon\text{-closure} \left(\delta \left(\hat{\delta}(q_2, \varepsilon), 2 \right) \right) \\
&= \varepsilon\text{-closure} \left(\delta(\varepsilon\text{-closure}(q_2), 2) \right) \\
&= \varepsilon\text{-closure} (\delta(q_2, 2)) \\
&= \varepsilon\text{-closure} (q_2) \\
&= \{q_2\}
\end{aligned}$$

Now we will summarize all the computed δ' transitions -

$$\delta'(q_0, 0) = \{q_0, q_1, q_2\}$$

$$\delta'(q_0, 1) = \{q_1, q_2\}$$

$$\delta'(q_0, 2) = \{q_2\}$$

$$\delta'(q_1, 0) = \phi,$$

$$\delta'(q_1, 1) = \{q_1, q_2\}$$

$$\delta'(q_1, 2) = \{q_2\}$$

$$\delta'(q_2, 0) = \phi,$$

$$\delta'(q_2, 1) = \phi,$$

$$\delta'(q_2, 2) = \{q_2\}$$

From this we can write the transition table as -

Input \ State	0	1	2
q_0	$\{q_0, q_1, q_2\}$	$\{q_1, q_2\}$	$\{q_2\}$
q_1	ϕ	$\{q_1, q_2\}$	$\{q_2\}$
q_2	ϕ	ϕ	$\{q_2\}$

The NFA will be as shown in Fig. 1.12.2.

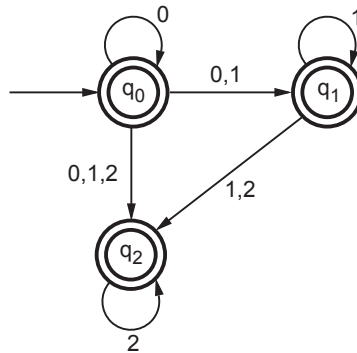


Fig. 1.12.2

Here q_0 , q_1 and q_2 is a final state because ϵ - closure (q_0), ϵ - closure (q_1) and ϵ - closure (q_2) contains final state q_2 .

Example 1.12.2 Convert the following NFA with ϵ to NFA without ϵ .

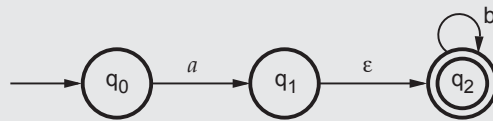


Fig. 1.12.3

Solution : We will first obtain ϵ - closures of q_0 , q_1 and q_2 as follows.

$$\epsilon - \text{closure}(q_0) = \{q_0\}$$

$$\epsilon - \text{closure}(q_1) = \{q_1, q_2\}$$

$$\epsilon - \text{closure}(q_2) = \{q_2\}$$

Now the δ' transitions on each input symbol is obtained as

$$\begin{aligned}
 \delta'(q_0, a) &= \epsilon - \text{closure}\left(\delta\left(\hat{\delta}(q_0, \epsilon), a\right)\right) \\
 &= \epsilon - \text{closure}\left(\delta\left(\epsilon - \text{closure}(q_0), a\right)\right) \\
 &= \epsilon - \text{closure}\left(\delta(q_0, a)\right) \\
 &= \epsilon - \text{closure}(q_1) \\
 &= \{q_1, q_2\} \\
 \delta'(q_0, b) &= \epsilon - \text{closure}\left(\delta\left(\hat{\delta}(q_0, \epsilon), b\right)\right) \\
 &= \epsilon - \text{closure}\left(\delta\left(\epsilon - \text{closure}(q_0), b\right)\right)
 \end{aligned}$$

$$= \varepsilon - \text{closure}(\delta(q_0, b))$$

$$= \phi$$

$$\delta'(q_1, a) = \varepsilon - \text{closure}(\delta(\hat{\delta}(q_1, \varepsilon), a))$$

$$= \varepsilon - \text{closure}(\delta(\varepsilon - \text{closure}(q_1), a))$$

$$= \varepsilon - \text{closure}(\delta(q_1, q_2), a)$$

$$= \varepsilon - \text{closure}(\delta(q_1, a) \cup \delta(q_2, a))$$

$$= \varepsilon - \text{closure}(\phi \cup \phi)$$

$$= \phi$$

$$\delta'(q_1, b) = \varepsilon - \text{closure}(\delta(\hat{\delta}(q_1, \varepsilon), b))$$

$$= \varepsilon - \text{closure}(\delta(\varepsilon - \text{closure}(q_1), b))$$

$$= \varepsilon - \text{closure}(\delta(q_1, q_2), b)$$

$$= \varepsilon - \text{closure}(\delta(q_1, b) \cup \delta(q_2, b))$$

$$= \varepsilon - \text{closure}(\phi \cup q_2)$$

$$= \varepsilon - \text{closure}(q_2)$$

$$= \{q_2\}$$

$$\delta'(q_2, a) = \varepsilon - \text{closure}(\delta(\hat{\delta}(q_2, \varepsilon), a))$$

$$= \varepsilon - \text{closure}(\delta(\varepsilon - \text{closure}(q_2), a))$$

$$= \varepsilon - \text{closure}(\delta(q_2, a))$$

$$= \varepsilon - \text{closure}(\phi)$$

$$= \phi$$

$$\delta'(q_2, b) = \varepsilon - \text{closure}(\delta(\hat{\delta}(q_2, \varepsilon), b))$$

$$= \varepsilon - \text{closure}(\delta(\varepsilon - \text{closure}(q_2), b))$$

$$= \varepsilon - \text{closure}(\delta(q_2, b))$$

$$= \varepsilon - \text{closure}(q_2)$$

$$= \{q_2\}$$

The transition table can be -

→

State \ Input	a	b
q ₀	{q ₁ , q ₂ }	ϕ
q ₁	ϕ	{q ₂ }
q ₂	ϕ	{q ₂ }

States q₁ and q₂ becomes the final as ϵ - closures of q₁ and q₂ contains the final state q₂. The NFA can be shown by following transition diagram -

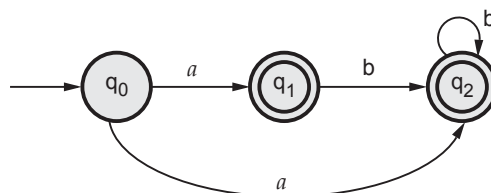


Fig. 1.12.4

Example 1.12.3 Convert the following NFA with ϵ -moves into NFA without ϵ moves.

SPPU : May-19, End Sem, Marks 6

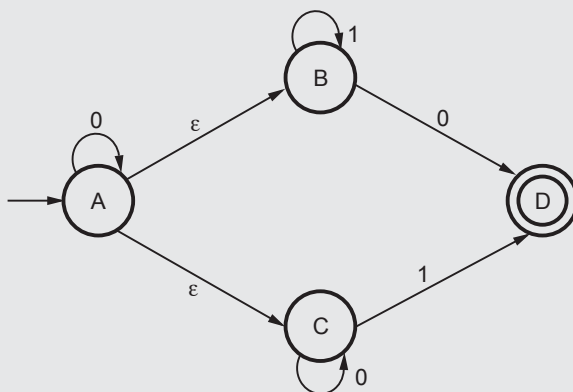


Fig. 1.12.5

Solution : We will compute ϵ closures of A, B, C, D

$$\epsilon - \text{closure}(A) = \{A, B, C\}$$

$$\epsilon - \text{closure}(B) = \{B\}$$

$$\epsilon - \text{closure}(C) = \{C\}$$

$$\epsilon - \text{closure}(D) = \{D\}$$

The δ' transitions on each input symbol is

$$\begin{aligned}
\delta' (A,O) &= \varepsilon - \text{closure } (\delta (\varepsilon - \text{closure } (A,O))) \\
&= \varepsilon - \text{closure } (\delta (A, B, C), O) \\
&= \varepsilon - \text{closure } (\delta (A,O) \cup \delta (B,O) \cup \delta (C,O)) \\
&= \varepsilon - \text{closure } (A, D, C) \\
&= \{A,B,C\} \cup \{D\}, \{C\}
\end{aligned}$$

$$\delta' (A,O) = \{A,B,C,D\}$$

$$\begin{aligned}
\delta' (A,1) &= \varepsilon - \text{closure } (\delta (\varepsilon - \text{closure } (A),1)) \\
&= \varepsilon - \text{closure } (\delta (A,B,C),1) \\
&= \varepsilon - \text{closure } (\delta (A,1) \cup \delta (B, 1) \cup \delta (C, 1)) \\
&= \varepsilon - \text{closure } (\phi \cup B \cup D) \\
&= \varepsilon - \text{closure } (B \cup D)
\end{aligned}$$

$$\delta' (A,1) = \{B, D\}$$

$$\begin{aligned}
\delta' (B,0) &= \varepsilon - \text{closure } (\delta (\varepsilon - \text{closure } (B), 0)) \\
&= \varepsilon - \text{closure } (\delta (B, 0)) \\
&= \varepsilon - \text{closure } (D)
\end{aligned}$$

$$\delta' (B,0) = D$$

$$\begin{aligned}
\delta' (B,1) &= \varepsilon - \text{closure } (\delta (\varepsilon - \text{closure } (B),1)) \\
&= \varepsilon - \text{closure } (\delta (B, 1)) \\
&= \varepsilon - \text{closure } (B)
\end{aligned}$$

$$\delta' (B,1) = B$$

$$\begin{aligned}
\delta' (C,0) &= \varepsilon - \text{closure } (\delta (\varepsilon - \text{closure } (C),0)) \\
&= \varepsilon - \text{closure } (\delta (C, 0)) \\
&= \varepsilon - \text{closure } (C)
\end{aligned}$$

$$\delta' (C,0) = C$$

$$\begin{aligned}
\delta' (C,1) &= \varepsilon - \text{closure } (\delta (\varepsilon - \text{closure } (C),1)) \\
&= \varepsilon - \text{closure } (\delta (C, 1)) \\
&= \varepsilon - \text{closure } (D)
\end{aligned}$$

$$\delta' (C,1) = D$$

$$\delta' (D,1) = \varepsilon - \text{closure } (\delta (\varepsilon - \text{closure } (D),0))$$

$$= \varepsilon - \text{closure } (\delta (D, 0))$$

$$= \varepsilon - \text{closure } (\phi)$$

$$\delta' (D, 0) = \phi$$

$$\delta' (D, 1) = \varepsilon - \text{closure } (\delta (\varepsilon - \text{closure } (D, 1))$$

$$= \varepsilon - \text{closure } (\phi)$$

$$\delta' (D, 1) = \phi$$

The transition table for NFA without ε is

State \ i/p	0	1
A	{A, B, C, D}	{B, D}
B	{D}	{B}
C	{C}	{D}
D	ϕ	ϕ

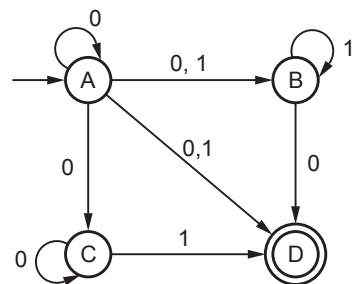


Fig. 1.12.6

1.13 NFA to DFA Conversion

SPPU : Dec.-05,09,13,17, May-09,10,12, Aug.-15,17, Marks 10

We will discuss the method of converting NFA to its equivalent DFA.

Let, $M = (Q, \Sigma, \delta, q_0, F)$ is a NFA which accepts the language $L(M)$. There should be equivalent DFA denoted by $M' = (Q', \Sigma', \delta', q'_0, F')$ such that $L(M) = L(M')$.

The conversion method will follow following steps -

- 1) The start state of NFA M will be the start for DFA M' . Hence add q_0 of NFA (start state) to Q' . Then find the transitions from this start state.
- 2) For each state $[q_1, q_2, q_3, \dots, q_i]$ in Q' the transitions for each input symbol Σ can be obtained as,
 - i) $\delta'([q_1, q_2, \dots, q_i], a) = \delta(q_1, a) \cup \delta(q_2, a) \cup \dots \cup \delta(q_i, a)$
 $= [q_1, q_2, \dots, q_k]$ may be some state.
 - ii) Add the state $[q_1, q_2, \dots, q_k]$ to DFA if it is not already added in Q' .
 - iii) Then find the transitions for every input symbol from Σ for state $[q_1, q_2, \dots, q_k]$. If we get some state $[q_1, q_2, \dots, q_n]$ which is not in Q' of DFA then add this state to Q' .
 - iv) If there is no new state generating then stop the process after finding all the transitions.

- 3) For the state $[q_1, q_2, \dots, q_n] \in Q'$ of DFA if any one state q_i is a final state of NFA then $[q_1, q_2, \dots, q_n]$ becomes a final state. Thus the set of all the final states $\in F'$ of DFA.

Example 1.13.1 Convert the given NFA to DFA.

State \ Input	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	q_0
q_1	q_2	q_1
q_2	q_3	q_3
q_3	ϕ	ϕ

Solution : As we know, first we will compute δ' function.

$$\delta(\{q_0\}, 0) = \{q_0, q_1\}$$

$$\text{Hence } \delta'([q_0], 0) = [q_0, q_1]$$

Similarly,

$$\delta(\{q_0\}, 1) = \{q_0\}$$

$$\text{Hence } \delta'([q_0], 1) = [q_0]$$

Thus we have got a new state $[q_0, q_1]$.

Let us check how it behaves on input 0 and 1.

So,

$$\begin{aligned} \delta'([q_0, q_1], 0) &= \delta([q_0], 0) \cup \delta([q_1], 0) \\ &= \{q_0, q_1\} \cup \{q_2\} \\ &= \{q_0, q_1, q_2\} \end{aligned}$$

Hence a new state is generated i.e. $[q_0, q_1, q_2]$

Similarly,

$$\begin{aligned} \delta'([q_0, q_1], 1) &= \delta([q_0], 1) \cup \delta([q_1], 1) \\ &= \{q_0\} \cup \{q_1\} \\ &= \{q_0, q_1\} \end{aligned}$$

No new state is generated here.

Again δ' function will be computed for $[q_0, q_1, q_2]$, the new state being generated.

State \ Input	0	1
$[q_0]$	$[q_0, q_1]$	$[q_0]$
$[q_0, q_1]$	$[q_0, q_1, q_2]$	$[q_0, q_1]$
$[q_0, q_1, q_2]$	$[q_0, q_1, q_2, q_3]$	$[q_0, q_1, q_3]$

As, you have observed in the above table for a new state $[q_0, q_1, q_2]$ the input 0 will give a new state $[q_0, q_1, q_2, q_3]$ and input 1 will give a new state $[q_0, q_1, q_3]$, because

$$\begin{aligned}
 \delta'([q_0, q_1, q_2], 0) &= \delta'([q_0], 0) \cup \delta'([q_1], 0) \cup \delta'([q_2], 0) \\
 &= \{q_0, q_1\} \cup \{q_2\} \cup \{q_3\} \\
 &= \{q_0, q_1, q_2, q_3\} \\
 &= [q_0, q_1, q_2, q_3]
 \end{aligned}$$

Same procedure for input 1. Thus the final DFA is as given below.

State	0	1
$\rightarrow [q_0]$	$[q_0, q_1]$	$[q_0]$
$[q_1]$	$[q_2]$	$[q_1]$
$[q_2]$	$[q_3]$	$[q_3]$
$[q_3]$	ϕ	$[q_2]$
$[q_0, q_1]$	$[q_0, q_1, q_2]$	$[q_0, q_1]$
$[q_0, q_1, q_2]$	$[q_0, q_1, q_2, q_3]$	$[q_0, q_1, q_3]$
$[q_0, q_1, q_3]$	$[q_0, q_1, q_2]$	$[q_0, q_1, q_2]$
$[q_0, q_1, q_2, q_3]$	$[q_0, q_1, q_2, q_3]$	$[q_0, q_1, q_2, q_3]$

DFA for example 1.13.1

Example 1.13.2 Construct DFA equivalent to the given NFA.**SPPU : Dec.-05, Marks 10**

State \ Input	0	1
$\rightarrow p$	{p, q}	p
q	r	r
r	s	–
s	s	s

Solution : The NFA $M = \{p, q, r, s\}, \{0, 1\}, \delta, \{p\}, \{s\}$

The equivalent DFA will be constructed.

State \ Input	0	1
$\rightarrow [p]$	[p, q]	[p]
[q]	[r]	[r]
[r]	[s]	–
([s])	[s]	[s]
[p, q]	[p, q, r]	[p, r]

Continuing with the generated new states.

State \ Input	0	1
$\rightarrow [p]$	[p, q]	[p]
[q]	[r]	[r]
[r]	[s]	–
([s])	[s]	[s]
[p, q]	[p, q, r]	[p, r]
[p, q, r]	[p, q, r, s]	[p, r]
[p, r]	[p, q, s]	[p]

$[p, q, r, s]$	$[p, q, r, s]$	$[p, r, s]$
$[p, q, s]$	$[p, q, r, s]$	$[p, r, s]$
$[p, r, s]$	$[p, q, s]$	$[p, s]$
$[p, s]$	$[p, q, s]$	$[p, s]$

The final state $F' = \{ [s], [p, q, r, s], [p, q, s], [p, r, s], [p, s] \}$

The transition graph shows two disconnected parts. But part I will be accepted as final DFA because it consists of start state and final states, in part II there is no start state.

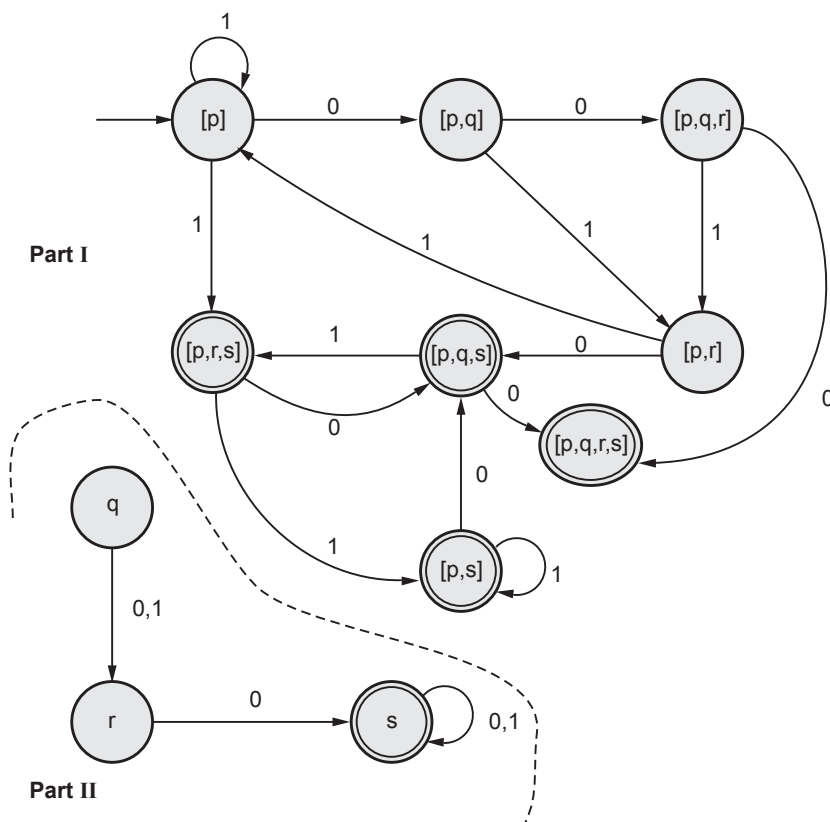


Fig. 1.13.1

Example 1.13.3 Convert the given NFA to DFA.

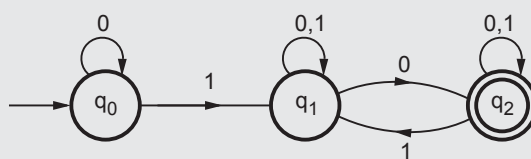


Fig. 1.13.2

Solution : For the given transition diagram in problem statement we will first construct the transition table.

→

State / Σ	0	1
q_0	q_0	q_1
q_1	$\{q_1, q_2\}$	q_1
$\textcircled{q_2}$	$\{q_2\}$	$\{q_1, q_2\}$

Now we will obtain δ' transitions for each state.

$$\delta'([q_0], 0) = \{q_0\}$$

$$= [q_0]$$

$$\delta'([q_0], 1) = \{q_1\}$$

$$= [q_1]$$

$$\delta'([q_1], 0) = \{q_1, q_2\}$$

$$= [q_1, q_2] \rightarrow \text{new state generated.}$$

$$\delta'([q_1], 1) = \{q_1\} = [q_1]$$

$$\delta'([q_2], 0) = \{q_2\}$$

$$= [q_2]$$

$$\delta'([q_2], 1) = \{q_1, q_2\}$$

$$= [q_1, q_2]$$

Now we will obtain δ' transitions on $[q_1, q_2]$.

$$\delta'([q_1, q_2], 0) = \delta([q_1], 0) \cup \delta([q_2], 0)$$

$$= \{q_1, q_2\} \cup \{q_2\}$$

$$= \{q_1, q_2\}$$

$$= [q_1, q_2]$$

$$\delta'([q_1, q_2], 1) = \delta([q_1], 1) \cup \delta([q_2], 1)$$

$$= \{q_1\} \cup \{q_1, q_2\}$$

$$= [q_1, q_2]$$

The state $[q_1, q_2]$ is final state as well because it contains a final state q_2 . The transition table for the constructed DFA will be -

→

State / Σ	0	1
$[q_0]$	$[q_0]$	$[q_1]$
$[q_1]$	$[q_1, q_2]$	$[q_1]$
$[q_2]$	$[q_2]$	$[q_1, q_2]$
$[q_1, q_2]$	$[q_1, q_2]$	$[q_1, q_2]$

The transition diagram will be -

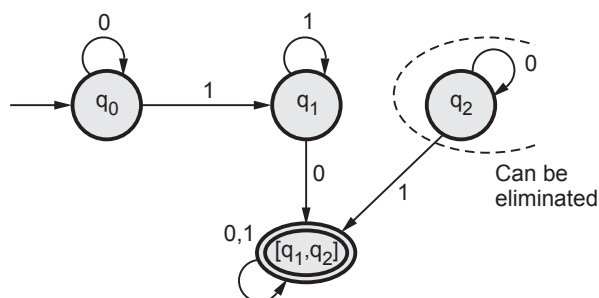


Fig. 1.13.3

Consider a string 0010 we will get -

Using NFA

q_0 0010 $\vdash$ $0q_0$ 0110
 $\vdash$ $00q_0$ 110
 $\vdash$ $001q_1$ 10
 $\vdash$ $0011q_1$ 0
 $\vdash$ $00110q_2$ or $\vdash$ $00110q_1$
 Accept non-accept.

Using DFA

q_0 00110 $\vdash$ $0q_0$ 0110
 $\vdash$ $00q_0$ 110
 $\vdash$ $001q_1$ 10

$\vdash 0011q_10$

$\vdash 00110 [q_1, q_2]$

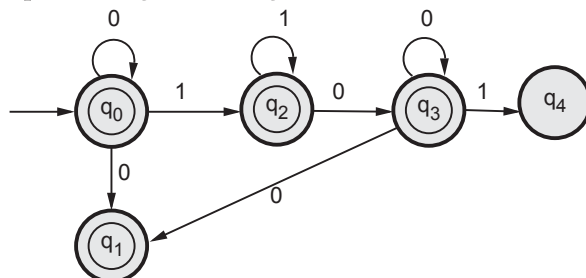
Accept state.

Thus both the NFA and DFA accept the same string 00110.

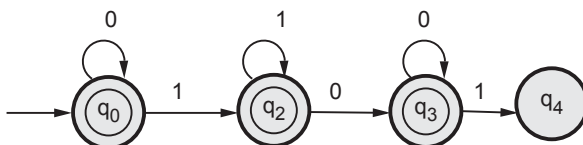
Example 1.13.4 Construct a NFA and then equivalent DFA accepting strings over $\{0,1\}$, for accepting all possible strings of zeroes and ones not containing 101 as a substring.

SPPU : May-09; Dec.-09, Marks 10

Solution : The NFA representing the strings that do not contain 101 as a substring is



The equivalent DFA will be

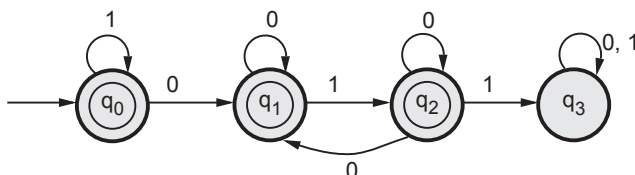


Example 1.13.5 Construct a NFA and then equivalent DFA accepting string over $\{0, 1\}$, for accepting all possible strings of zeroes and ones which does not contain 011 as a substring.

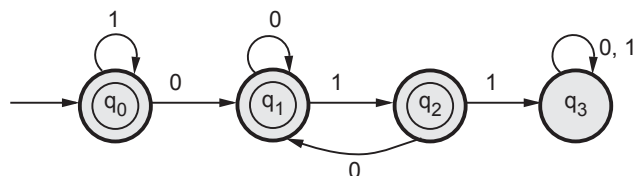
SPPU : May-09, 10, 12, Dec.-09, Marks 10

Solution :

NFA will be



DFA will be



Example 1.13.6 Convert given non-deterministic finite automata (NFA) to deterministic finite automata (DFA). Informally describe the language it accepts.

SPPU : Dec.-13, Marks 6

δ	0	1
p	{p, q}	{p}
q	ϕ	{r}
*r	{p, r}	{q}

Solution :

$$\delta(p, 0) = \{p, q\} \rightarrow \mathbf{[p, q]} \text{ new state}$$

$$\delta(p, 1) = \{p\} = [p] \text{ state}$$

$$\delta(q, 0) = \phi$$

$$\delta(q, 1) = \{r\} = [r] \text{ state}$$

$$\delta(r, 0) = \{p, r\} = \mathbf{[p, r]} \text{ new state}$$

$$\delta(r, 1) = \{q\} = [q] \text{ state}$$

Now, we will find input transitions on newly generated states.

$$\delta([p, q], 0) = [p, q]$$

$$\delta([p, q], 1) = [p, r]$$

$$\delta([p, r], 0) = \{p, q, r\} = \mathbf{[p, q, r]} \text{ new state}$$

$$\delta([p, r], 1) = \{p, q\} = \mathbf{[p, q]} \text{ new state}$$

$$\delta([p, q, r], 0) = [p, q, r]$$

$$\delta([p, q, r], 1) = [p, q, r]$$

The transition table for NFA is -

		0	1
→	[p]	[p, q]	[p]
	[q]	ϕ	[r]
*	[r]	[p, r]	[q]
	[p, q]	[p, q]	[p, r]
*	[p, r]	[p, q, r]	[p, q]
*	[p, q, r]	[p, q, r]	[p, q, r]

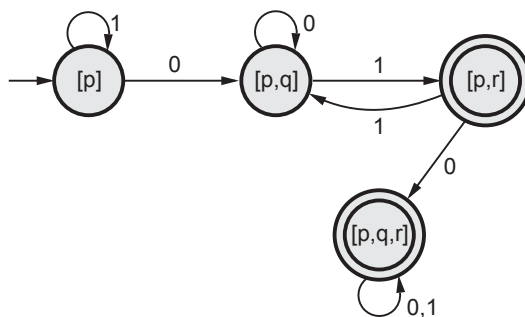


Fig. 1.13.4

Example 1.13.7 Convert following NFA into its equivalent DFA.

SPPU : Aug.-15, In Sem, Marks 4

Q \ $\Sigma \rightarrow$	0	1
$\rightarrow P$	P, Q	R
Q	R	R
R	S	Q
* S	S	S

Solution :

$$\delta(P, 0) = \{P, Q\} = [P, Q] \text{ New state}$$

$$\delta(P, 1) = R$$

$$\delta(Q, 0) = R$$

$$\delta(Q, 1) = R$$

$$\delta(R, 0) = S$$

$$\delta(R, 1) = Q$$

$$\delta(S, 0) = S$$

$$\delta(S, 1) = S$$

The δ transitions can be applied on new state $[P, Q]$

$$\therefore \delta([P, Q], 0) = \{P, Q, R\} = [P, Q, R] \text{ New state}$$

$$\delta([P, Q], 1) = R$$

$$\delta([P, Q, R], 0) = \{P, Q, R, S\} = [P, Q, R, S] \text{ New state}$$

$$\delta([P, Q, R], 1) = \{R, Q\} = [R, Q] \text{ New state}$$

$$\delta([P, Q, R, S], 0) = [P, Q, R, S]$$

$$\delta([P, Q, R, S], 1) = \{R, Q, S\} = [R, Q, S] \text{ New state}$$

$$\delta([R, Q], 0) = \{R, S\} = [R, S] \text{ New state}$$

$$\delta([R, Q], 1) = [R, Q]$$

$$\delta([R, Q, S], 0) = [R, S]$$

$$\delta([R, Q, S], 1) = [R, Q, S]$$

$$\delta([R, S], 0) = S$$

$$\delta([R, S], 1) = [Q, S] \text{ New state}$$

$$\delta([Q, S], 0) = [R, S]$$

$$\delta([Q, S], 1) = [R, S]$$

As no new state is getting generated we will stop applying δ transitions. The transition table for DFA is as follows.

Input State		0	1
→	[P]	[P, Q]	[R]
	[Q]	[R]	[R]
	[R]	[S]	[Q]
*	[S]	[S]	[S]
	[P, Q]	[P, Q, R]	[R]
	[P, Q, R]	[P, Q, R, S]	[R, Q]
*	[P, Q, R, S]	[P, Q, R, S]	[R, Q, S]
	[R, Q]	[R, S]	[R, Q]
*	[R, Q, S]	[R, S]	[R, Q, S]
*	[R, S]	[S]	[Q, S]
*	[Q, S]	[R, S]	[R, S]

Example 1.13.8 Construct an equivalent DFA for the following NFA :

SPPU : Aug.-17, In Sem, Marks 4

States/ Σ	0	1
→ p	{p, q}	{q}
Ⓚ	{r}	{r}
r	—	{r}

Solution : Let us apply δ transitions on each state using each input symbol.

$$\delta(p, 0) = \{p, q\} = [p, q] \text{ new state}$$

$$\delta(p, 1) = \{q\}$$

$$\delta([p, q], 0) = \{p, q, r\} = [p, q, r] \text{ new state}$$

$$\delta([p, q], 1) = \{q, r\} = [q, r] \text{ new state}$$

$$\delta([p, q, r], 0) = [p, q, r]$$

$$\delta([p, q, r], 1) = [q, r]$$

$$\delta([q, r], 0) = [r]$$

$$\delta([q, r], 1) = [r]$$

As no new states are getting generated. There are no δ transitions.

The transition table will be

State/ Σ	0	1
[p]	[p, q]	[q]
* [p, q]	[p, q, r]	[q, r]
* [p, q, r]	[p, q, r]	[q, r]
* [q, r]	[r]	[r]
* [q]	[r]	[r]
[r]	-	[r]

Here r state can be eliminated

The DFA will be

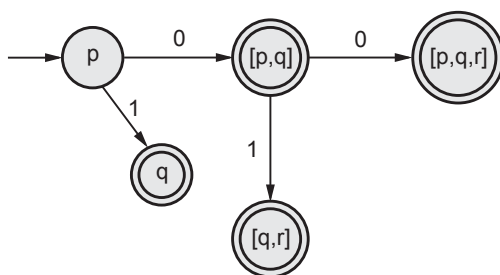


Fig. 1.13.5

Example 1.13.9 Let $M = \{\{q_0, q_1\}, \{0, 1\}, \delta, q_0, \{q_1\}\}$ be an NFA

Where $\delta(q_0, 0) = \{q_0, q_1\}$

$$\delta(q_0, 1) = \{q_1\}$$

$$\delta(q_1, 0) = \phi$$

$$\delta(q_1, 1) = \{q_0, q_1\}$$

Construct an equivalent DFA.

SPPU : Dec.-17, End Sem, Marks 4

Solution : Let us assume δ function as follows :

$$\delta(q_0, 0) = \{q_0, q_1\}$$

$$\delta(q_0, 1) = \{q_1\}$$

$$\delta(q_1, 0) = \phi$$

$$\delta(q_1, 1) = \{q_0, q_1\}$$

Let us apply δ transition on $\{q_0, q_1\}$ state

$$\delta([q_0, q_1], 0) = [q_0, q_1]$$

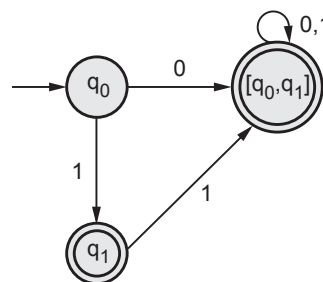
$$\delta([q_0, q_1], 1) = [q_0, q_1]$$

As there are no new states getting generated, there is no application of δ transition. Hence the transition table and transition diagram for equivalent DFA is

i/p \ State	0	1
[q ₀]	[q ₀ , q ₁]	[q ₁]
[q ₀ , q ₁]	[q ₀ , q ₁]	[q ₀ , q ₁]
[q ₁]	ϕ	[q ₀ , q ₁]

i/p \ State	0	1
q ₀	{q ₀ , q ₁ }	{q ₁ }
q ₁	ϕ	{q ₀ , q ₁ }

Transition table



1.14 NFA with Epsilon to DFA Conversion

SPPU : Dec.-11, Marks 6

Subset Construction Method Algorithm

Step 1 : Initialization step

Let, ε - closure (s) be the states in Dstates of DFA.

Step 2 : Repeatability step

```

While (T are the unmarked states in Dstates)
{
    mark T
    for (each symbol a)
    {
        U =  $\varepsilon$  - closure (move (T, a))
        if (U is not in Dstates)
            Add U as unmarked state in Dstates
        Dtran [T, a] = U
    } // end of for
} // end of while.
  
```

Example 1.14.1 Convert the following NFA with ϵ to equivalent DFA.

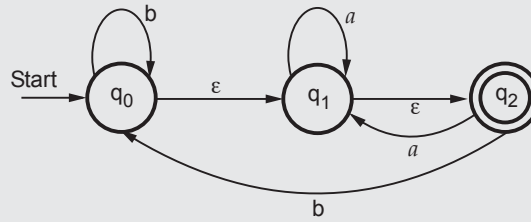


Fig. 1.14.1

Solution : To convert this NFA we will first find ϵ -closures.

$$\epsilon\text{-closures } \{q_0\} = \{q_0, q_1, q_2\}$$

$$\epsilon\text{-closures } \{q_1\} = \{q_1, q_2\}$$

$$\epsilon\text{-closures } \{q_2\} = \{q_2\}$$

Let us start from ϵ -closure of start state

$$\epsilon\text{-closure } \{q_0\} = \{q_0, q_1, q_2\} \text{ we will call this state as A.}$$

Now let us find transitions on A with every input symbol.

$$\begin{aligned}
 \delta'(A, a) &= \epsilon\text{-closure } (\delta(A, a)) \\
 &= \epsilon\text{-closure } (\delta(q_0, a) \cup \delta(q_1, a) \cup \delta(q_2, a)) \\
 &= \epsilon\text{-closure } \{\delta(q_0, a) \cup \delta(q_1, a) \cup \delta(q_2, a)\} \\
 &= \epsilon\text{-closure } \{q_1\} \\
 &= \{q_1, q_2\}. \text{ Let us call it as state B.}
 \end{aligned}$$

$$\begin{aligned}
 \delta'(A, b) &= \epsilon\text{-closure } (\delta(A, b)) \\
 &= \epsilon\text{-closure } (\delta(q_0, b) \cup \delta(q_1, b) \cup \delta(q_2, b)) \\
 &= \epsilon\text{-closure } \{\delta(q_0, b) \cup \delta(q_1, b) \cup \delta(q_2, b)\} \\
 &= \epsilon\text{-closure } \{q_0\} \\
 &= \{q_0, q_1, q_2\} \text{ i.e. A.}
 \end{aligned}$$

Hence we can state that

$$\delta'(A, a) = B$$

$$\delta'(A, b) = A$$

Now let us find transitions for state $B = \{q_1, q_2\}$.

$$\delta'(B, a) = \epsilon\text{-closure } (\delta(q_1, a) \cup \delta(q_2, a))$$

$$\begin{aligned}
 &= \varepsilon - \text{closure } \{q_1\} \\
 &= \{q_1, q_2\} \text{ i.e. B} \\
 \delta'(B, b) &= \varepsilon - \text{closure } (\delta(q_1, b)) \\
 &= \varepsilon - \text{closure } \{\delta(q_1, b) \cup \delta(q_2, b)\} \\
 &= \varepsilon - \text{closure } \{q_0\} \\
 &= \{q_0, q_1, q_2\} \text{ i.e. A.}
 \end{aligned}$$

Hence the generated DFA is

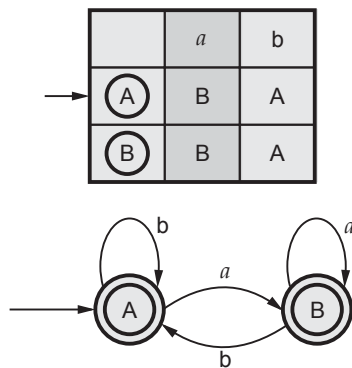


Fig. 1.14.2

Example 1.14.2 Convert the given NFA into its equivalent DFA -

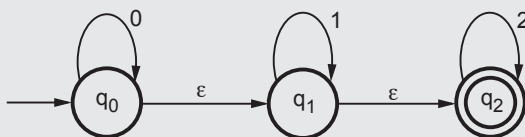


Fig. 1.14.3

Solution : Let us obtain ε - closure of each state.

$$\varepsilon - \text{closure } (q_0) = \{q_0, q_1, q_2\}$$

$$\varepsilon - \text{closure } (q_1) = \{q_1, q_2\}$$

$$\varepsilon - \text{closure } (q_2) = \{q_2\}$$

Now we will obtain δ' transition. Let $\varepsilon - \text{closure } (q_0) = \{q_0, q_1, q_2\}$ call it as **state A**.

$$\begin{aligned}
 \delta'(A, 0) &= \varepsilon - \text{closure } \{\delta((q_0, q_1, q_2), 0)\} \\
 &= \varepsilon - \text{closure } \{\delta(q_0, 0) \cup \delta(q_1, 0) \cup \delta(q_2, 0)\} \\
 &= \varepsilon - \text{closure } \{q_0\} \\
 &= \{q_0, q_1, q_2\} \text{ i.e. state A}
 \end{aligned}$$

$$\begin{aligned}
\delta'(A,1) &= \varepsilon\text{-closure } \{\delta((q_0, q_1, q_2), 1)\} \\
&= \varepsilon\text{-closure } \{\delta(q_0,1) \cup \delta(q_1,1) \cup \delta(q_2,1)\} \\
&= \varepsilon\text{-closure } \{q_1\} \\
&= \{q_1, q_2\}. \text{ Call it as } \mathbf{state\ B} \\
\delta'(A,2) &= \varepsilon\text{-closure } \{\delta((q_0, q_1, q_2), 2)\} \\
&= \varepsilon\text{-closure } \{\delta(q_0,2) \cup \delta(q_1,2) \cup \delta(q_2,2)\} \\
&= \varepsilon\text{-closure } \{q_2\} \\
&= \{q_2\}. \text{ Call it as } \mathbf{state\ C}.
\end{aligned}$$

Thus we have obtained

$$\delta'(A,0) = A$$

$$\delta'(A,1) = B$$

$$\delta'(A,2) = C$$

i.e.

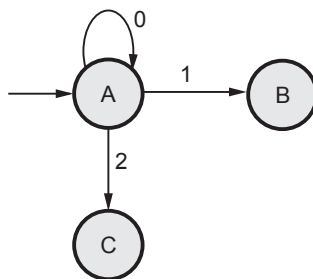


Fig. 1.14.4

Now we will find transitions on states B and C for each input.

Hence

$$\begin{aligned}
\delta'(B,0) &= \varepsilon\text{-closure } \{\delta(q_1, q_2), 0\} \\
&= \varepsilon\text{-closure } \{\delta(q_1,0) \cup \delta(q_2,0)\} \\
&= \varepsilon\text{-closure } \{\phi\} \\
&= \phi \\
\delta'(B,1) &= \varepsilon\text{-closure } \{\delta(q_1, q_2), 1\} \\
&= \varepsilon\text{-closure } \{\delta(q_1,1) \cup \delta(q_2,1)\} \\
&= \varepsilon\text{-closure } \{q_1\}
\end{aligned}$$

$$= \{q_1, q_2\} \text{ i.e. state B itself.}$$

$$\delta'(B, 2) = \varepsilon\text{-closure} \{ \delta(q_1, 2), 2 \}$$

$$= \varepsilon\text{-closure} \{ \delta(q_1, 2) \cup \delta(q_2, 2) \}$$

$$= \varepsilon\text{-closure} \{ q_2 \}$$

$$= \{q_2\} \text{ i.e. state C.}$$

Hence

$$\delta'(B, 0) = \phi$$

$$\delta'(B, 1) = B$$

$$\delta'(B, 2) = C$$

The partial transition diagram will be

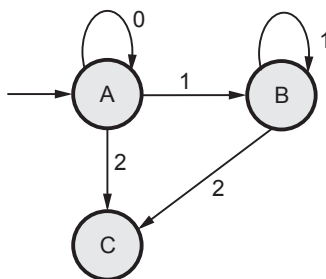


Fig. 1.14.5

Now we will obtain transitions for C :

$$\delta'(C, 0) = \varepsilon\text{-closure} \{ \delta(q_2, 0) \}$$

$$= \varepsilon\text{-closure} \{ \phi \}$$

$$= \phi$$

$$\delta'(C, 1) = \varepsilon\text{-closure} \{ \delta(q_2, 1) \}$$

$$= \varepsilon\text{-closure} \{ \phi \}$$

$$= \phi$$

$$\delta'(C, 2) = \varepsilon\text{-closure} \{ \delta(q_2, 2) \}$$

$$= q_2$$

Hence the DFA is

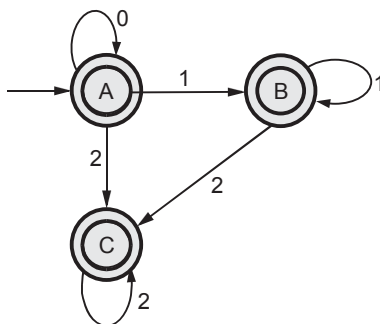


Fig. 1.14.6

As $A = \{q_0, q_1, q_2\}$ in which final state q_2 lies hence A is final state in $B = \{q_1, q_2\}$ the state q_2 lies hence B is also final state in $C = \{q_2\}$, the state q_2 lies hence C is also a final state.

Example 1.14.3 Convert the given NFA with ϵ to its equivalent DFA.

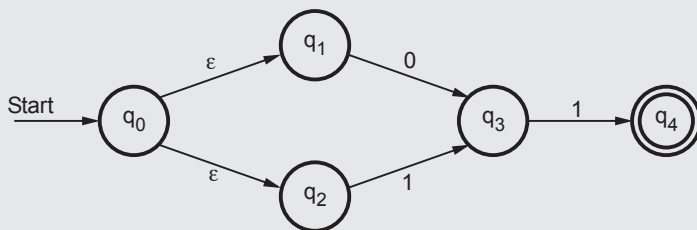


Fig. 1.14.7

Solution :

$$\epsilon - \text{closure } \{q_0\} = \{q_0, q_1, q_2\}$$

$$\epsilon - \text{closure } \{q_1\} = \{q_1\}$$

$$\epsilon - \text{closure } \{q_2\} = \{q_2\}$$

$$\epsilon - \text{closure } \{q_3\} = \{q_3\}$$

$$\epsilon - \text{closure } \{q_4\} = \{q_4\}$$

Now, Let $\epsilon - \text{closure } \{q_0\} = \{q_0, q_1, q_2\}$ be state A.

$$\text{Hence } \delta'(A, 0) = \epsilon - \text{closure } \{ \delta((q_0, q_1, q_2), 0) \}$$

$$= \epsilon - \text{closure } \{ \delta(q_0, 0) \cup \delta(q_1, 0) \cup \delta(q_2, 0) \}$$

$$= \epsilon - \text{closure } \{q_3\}$$

$$= \{q_3\} \text{ call it as state B.}$$

$$\begin{aligned}
 \delta'(A, 1) &= \varepsilon\text{-closure} \{ \delta((q_0, q_1, q_2), 1) \} \\
 &= \varepsilon\text{-closure} \{ \delta(q_0, 1) \cup \delta(q_1, 1) \cup \delta(q_2, 1) \} \\
 &= \varepsilon\text{-closure} \{ q_3 \} \\
 &= q_3 = B.
 \end{aligned}$$

The partial DFA will be

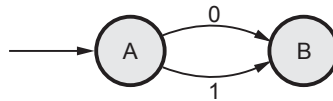


Fig. 1.14.8

Now,

$$\begin{aligned}
 \delta'(B, 0) &= \varepsilon\text{-closure} \{ \delta(q_3, 0) \} \\
 &= \phi \\
 \delta'(B, 1) &= \varepsilon\text{-closure} \{ \delta(q_3, 1) \} \\
 &= \varepsilon\text{-closure} \{ q_4 \} \\
 &= \{ q_4 \} \text{ i.e. state C} \\
 \delta'(C, 0) &= \varepsilon\text{-closure} \{ \delta(q_4, 0) \} \\
 &= \phi \\
 \delta'(C, 1) &= \varepsilon\text{-closure} \{ \delta(q_4, 1) \} \\
 &= \phi
 \end{aligned}$$

The DFA will be,

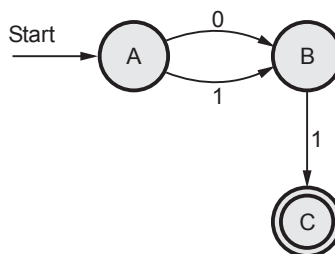


Fig. 1.14.9

Example 1.14.4 Convert the given NFA- $\wedge$ to an NFA.

SPPU : Dec.-11, Marks 6

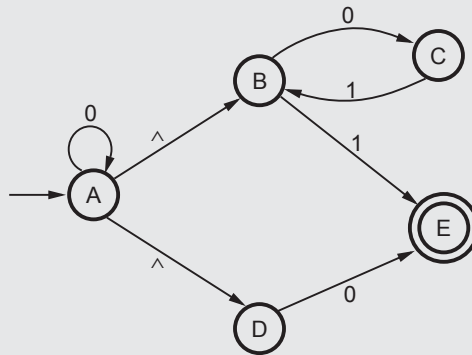


Fig. 1.14.10

Solution : We will ϵ -closures of A, B, C, D and E.

$$\epsilon\text{-closure}(A) = \{A, B, D\}$$

$$\epsilon\text{-closure}(B) = \{B\}$$

$$\epsilon\text{-closure}(C) = \{C\}$$

$$\epsilon\text{-closure}(D) = \{D\}$$

$$\epsilon\text{-closure}(E) = \{E\}$$

The δ' transitions on each input symbol is -

$$\begin{aligned} \delta'(A, 0) &= \epsilon\text{-closure}(\delta(\epsilon\text{-closure}(A), 0)) \\ &= \epsilon\text{-closure}(\delta(\{A, B, D\}, 0)) \\ &= \epsilon\text{-closure}(A, C, E) \\ &= \epsilon\text{-closure}(A) \cup \epsilon\text{-closure}(C) \cup \epsilon\text{-closure}(E) \\ &= \{A, B, D\} \cup \{C\} \cup \{E\} \end{aligned}$$

$$\delta'(A, 0) = \{A, B, C, D, E\}$$

$$\begin{aligned} \delta'(A, 1) &= \epsilon\text{-closure}(\delta(\epsilon\text{-closure}(A), 1)) \\ &= \epsilon\text{-closure}(\delta(\{A, B, D\}, 1)) \\ &= \epsilon\text{-closure}(\{E\}) \end{aligned}$$

$$\delta'(A, 1) = \{E\}$$

$$\begin{aligned} \delta'(B, 0) &= \epsilon\text{-closure}(\delta(\epsilon\text{-closure}(B), 0)) \\ &= \epsilon\text{-closure}(\delta(\{B\}, 0)) \\ &= \epsilon\text{-closure}(C) \end{aligned}$$

$$\delta'(B, 0) = \{C\}$$

$$\delta'(B, 1) = \{E\}$$

$$\begin{aligned}\delta'(C, 0) &= \varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}(C), 0)) \\ &= \varepsilon\text{-closure}(\delta(C, 0))\end{aligned}$$

$$\delta'(C, 0) = \phi$$

$$\begin{aligned}\delta'(C, 1) &= \varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}(C), 1)) \\ &= \varepsilon\text{-closure}(\delta(C, 1)) \\ &= \varepsilon\text{-closure}(B)\end{aligned}$$

$$\delta'(C, 1) = \{B\}$$

$$\begin{aligned}\delta'(D, 0) &= \varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}(D), 0)) \\ &= \varepsilon\text{-closure}(\delta(D, 0)) \\ &= \varepsilon\text{-closure}(E) \\ &= \{E\}\end{aligned}$$

$$\begin{aligned}\delta'(D, 1) &= \varepsilon\text{-closure}(\delta(\varepsilon\text{-closure}(D), 1)) \\ &= \varepsilon\text{-closure}(\delta(D, 1)) \\ &= \varepsilon\text{-closure}(\phi)\end{aligned}$$

$$\delta'(D, 1) = \phi$$

$$\begin{aligned}\delta'(E, 0) &= \varepsilon\text{-closure}(\delta(E, 0)) \\ &= \varepsilon\text{-closure}(\phi)\end{aligned}$$

$$\delta'(E, 0) = \phi$$

Similarly, $\delta'(E, 1) = \phi$

The NFA can be represented by following transition table.

State \ i/p	0	1
A	{A, B, C, D, E}	E
B	C	E
C	ϕ	B
D	E	ϕ
E	ϕ	ϕ

→

1.15 Minimization of DFAs

SPPU : Dec.-12, Marks 8

- The minimization of FSM means reducing the number of states from given FA.
- While minimizing FSM we first find out which two states are equivalent we can represent those two states by one representative state.
- **Definition of equivalent states** - The two states q_1 and q_2 are equivalent if both $\delta(q_1, x)$ and $\delta(q_2, x)$ are final states or both of them are non final states for all $x \in \Sigma^*$ (Σ^* indicate any string of any length) we can minimize the given FSM by finding equivalent states.

Method for Construction of Minimum State Automata :

Step 1 : We will create a set π_0 as $\pi_0 = \{ Q_1^0, Q_2^0 \}$ where Q_1^0 is set of all final states

and $Q_2^0 = Q - Q_1^0$ where Q is a set of all the states in DFA.

Step 2 : Now we will construct π_{K+1} from π_K . Let Q_i^K be any subset in π_K . If q_1 and q_2 are in Q_i^K they are $(K + 1)$ equivalent provided $\delta(q_1, a)$ and $\delta(q_2, a)$ are K equivalent. Find out whether $\delta(q_1, a)$ and $\delta(q_2, a)$ are residing in the same equivalence class π_K . Then it is said that q_1 and q_2 are $(K + 1)$ equivalent. Thus from Q_i^K we create $(K + 1)$ equivalence classes. Repeat step 2 for every Q_i^K in π_K and obtain all the elements of π_{K+1} .

Step 3 : Construct π_n for $n = 1, 2, \dots$ until $\pi_n = \pi_{n+1}$.

Step 4 : Then replace all the equivalent states in one equivalence class by representative state. This helps in minimizing the given DFA.

Let us understand this method with the help of some examples.

Example 1.15.1 Construct the minimum state automaton for the following transition diagram.

SPPU : Dec.-12, Marks 8

Solution : We will first construct a transition table for the given DFA.

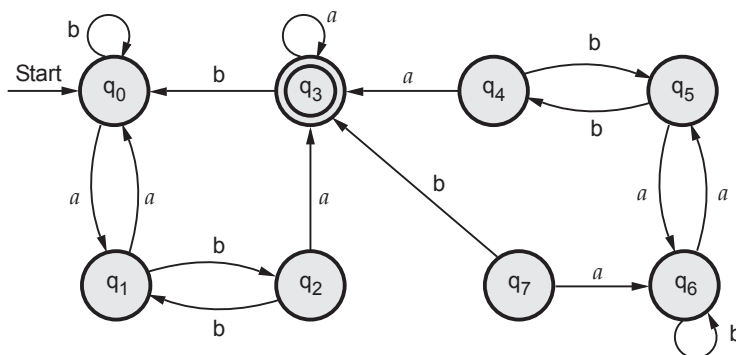


Fig. 1.15.1

We will start constructing equivalence classes.

State \ Input		
	a	b
q_0	q_1	q_0
q_1	q_0	q_2
q_2	q_3	q_1
q_3	q_3	q_0
q_4	q_3	q_5
q_5	q_6	q_4
q_6	q_5	q_6
q_7	q_6	q_3

Step 1 : We will first find 0 - equivalence. For that purpose we will create two sets - one set containing all the final states and other set containing all the non-final states.

$$\therefore \quad 0 - \text{Equivalent} = \{q_0, q_1, q_2, q_4, q_5, q_6, q_7\} \quad \{q_3\}$$

Now will check equivalence among all the states of $\{q_0, q_1, q_2, q_4, q_5, q_6, q_7\}$ by means of δ mapping functions

$$\begin{aligned} \delta(q_0, a) &= q_1 \\ \delta(q_1, a) &= q_0 \end{aligned}$$

↑
Belong to same set

$$\begin{aligned} \delta(q_0, b) &= q_0 \\ \delta(q_1, b) &= q_2 \end{aligned}$$

↑
Belong to same set

$\therefore q_0$ and q_1 are equivalent. In this way we will consider every pair from the set $\{q_0, q_1, q_2, q_4, q_5, q_6, q_7\}$

Now consider pair $\{q_0, q_2\}$

$$\begin{aligned} \delta(q_0, a) &= q_1 \\ \delta(q_2, a) &= q_3 \end{aligned}$$

← Belongs to different sets

$\therefore q_0$ and q_2 are not equivalent

Continuing in this way we get -

$$1 - \text{equivalence} = \{q_0, q_1, q_5, q_6\}, \{q_2, q_4\}, \{q_3\}, \{q_7\}$$

Step 2 : Now will find 2 - equivalence from the sets obtained in 1 - equivalence.

Consider the set $\{q_0, q_1, q_5, q_6\}$ first. We will compare q_0 with q_1, q_5, q_6 . Then we find

$$\begin{aligned}\delta(q_0, a) &= q_1 \\ \delta(q_1, a) &= q_0\end{aligned}$$

↑
Belong to same set

$$\begin{aligned}\delta(q_0, b) &= q_0 \\ \delta(q_1, b) &= q_2\end{aligned}$$

↑
Belong to different sets

$\therefore q_0$ and q_1 now does not belong to same set.

Now compare states q_0, q_5 states.

$$\delta(q_0, a) = q_1$$

$$\delta(q_5, a) = q_6$$

$$\begin{aligned}\delta(q_0, b) &= q_0 \\ \delta(q_5, b) &= q_4\end{aligned}$$

← Belongs to
different sets

$\therefore q_0, q_5$ are not equivalent. Now we will compare q_1 and q_5

$$\begin{aligned}\delta(q_1, a) &= q_0 \\ \delta(q_5, a) &= q_6\end{aligned}$$

↑
Belong to same set

$$\begin{aligned}\delta(q_1, b) &= q_2 \\ \delta(q_5, b) &= q_4\end{aligned}$$

↑
Belong to same set

That means - q_1 and q_5 are equivalent. Hence the 2 - equivalence set can be written as -

$$\boxed{2 - \text{equivalence} = \{q_0, q_6\}, \{q_1, q_5\}, \{q_2, q_4\}, \{q_3\}, \{q_7\}}$$

Step 3 : We will compare $\{q_0, q_6\}$

$$\begin{aligned}\delta(q_0, a) &= q_1 \\ \delta(q_6, a) &= q_5\end{aligned}$$

↑
Belong to same set

$$\begin{aligned}\delta(q_0, b) &= q_0 \\ \delta(q_6, b) &= q_6\end{aligned}$$

↑
Belong to same set

$\therefore \{q_0, q_6\}$ are equivalent. Similarly, we will compare $\{q_1, q_5\}, \{q_2, q_4\}$ and we find them as equivalent. Hence 3 - equivalence set is as follows -

$$\boxed{3 - \text{equivalence} = \{q_0, q_6\}, \{q_1, q_5\}, \{q_2, q_4\}, \{q_3\}, \{q_7\}}$$

As 2 - equivalence and 3 - equivalence are same sets. We will stop finding further equivalences.

Hence we can have minimized transition table and minimized DFA as follows -

		Input	
		a	b
→	[q ₀ , q ₆]	[q ₁ , q ₅]	[q ₀ , q ₆]
	[q ₁ , q ₅]	[q ₀ , q ₆]	[q ₂ , q ₄]
	[q ₂ , q ₄]	[q ₃]	[q ₁ , q ₅]
*	[q ₃]	[q ₃]	[q ₀ , q ₆]
	[q ₇]	[q ₀ , q ₆]	[q ₃]

The transition diagram with minimized states is -

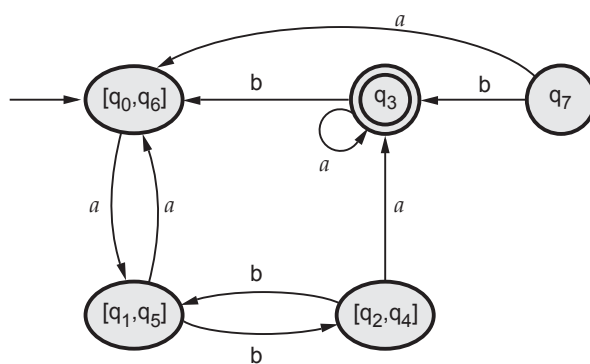


Fig. 1.15.2

Part III : FA with Output

1.16 Moore and Mealy Machines

SPPU : May-10,11,13,14,15, Dec.-06,11,13,14,19, Oct.-16, Marks 10

The finite automata is a collection of $(Q, \Sigma, \delta, q_0, F)$ where Q is a set of states including q_0 as a start state. In FA after reading the input string if we get final state then the string is said to be "acceptable". If we do not get final state then it is said that string is "rejected". That means there is no need of output for the finite Automata. The 'accept' or 'reject' acts like 'yes' or 'no' output for the machine. But if there is a need for specifying the output other than yes or no, then in such a case we require finite automata along with output. There are two types of FA with output and those are :

- 1) Moore machine
- 2) Mealy machine

Moore machine

Moore machine is a finite state machine in which the next state is decided by current state and current input symbol. The output symbol at a given time depends only on the present state of the machine. The formal definition of Moore machine is,

Moore machine is a six tuple $(Q, \Sigma, \Delta, \delta, \lambda, q_0)$ where

Q is finite set of states.

Σ is finite set of input symbols.

Δ is an output alphabet.

δ is an transition function such that $Q \times \Sigma \rightarrow Q$. This is also known as state function.

λ is output function $Q \rightarrow \Delta$. This function is also known as machine function.

q_0 is the initial state of machine.

For example

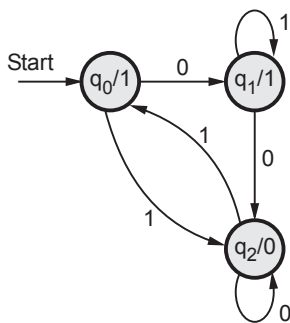


Fig. 1.16.1

Consider the Moore machine given below -

The transition table will be -

Current state	Next state (δ)		Output (λ)
	0	1	
q_0	q_1	q_2	1
q_1	q_2	q_1	1
q_2	q_2	q_0	0

In Moore machine output is associated with every state. In the above given Moore machine when machine is in q_0 state the output will be 1. For the Moore machine if the length of input string is n then output string has length $n + 1$.

For the string 0110 then the output will be 11110.

Mealy machine

Mealy machine is a machine in which output symbol depends upon the present input symbol and present state of the machine. The Mealy machine can be defined as -

Mealy machine is a six tuple $(Q, \Sigma, \Delta, \delta, \lambda, q_0)$ where

Q is finite set of states.

Σ is finite set of input symbols.

Δ is an output alphabet.

δ is state transition function such that $Q \times \Sigma \rightarrow Q$.

λ is machine function such that $Q \times \Sigma \rightarrow \Delta$.

q_0 is initial state of machine.

For example,

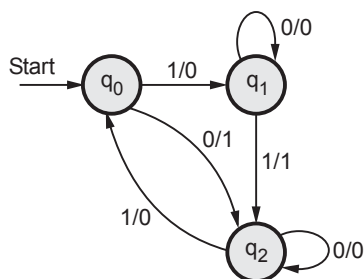


Fig. 1.16.2

For the input string 1001 the output will be 0001. In mealy machine the length of input string is equal to length of output string.

Difference between Moore and Mealy Machine

- 1) Length of Moore machine is one longer than Mealy machine for given input.
- 2) Secondly output of the input will be along the edges in case of Mealy machine but it should be associated with the state in case of Moore machine.

Problems Based on Moore and Melay Machine Modeling

Example 1.16.1 Design a Moore machine to generate 1's complement of given binary number.

SPPU : May-10, May-15, End Sem, Marks 6

Solution :

• Logic

To generate 1's complement of given binary number the simple logic which we will apply is that if input is 0 then output will be 1 and if input is 1 then output will be 0. That means there are three state one-start state, second state is for taking 0's as input and produces output as 1. Then third state is for taking 1's as input and producing output as 0.

• Design

Hence the Moore machine will be as shown in Fig. 1.16.3.

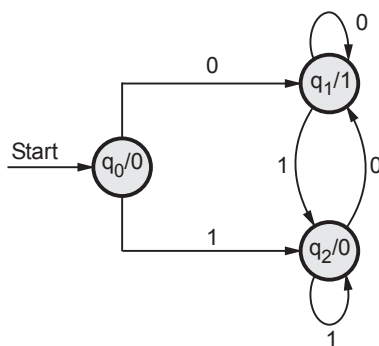


Fig. 1.16.3

• Simulation

For instance, take one binary number 1011 then

Input		1	0	1	1
State	q ₀	q ₂	q ₁	q ₂	q ₂
Output	0	0	1	0	0

Thus we get 00100 as 1's complement of 1011, we can neglect the initial 0 and the output which we get is 0100 which is 1's complement of 1011. The transition table can be drawn as below -

Current state	Next state		Output
	0	1	
q ₀	q ₁	q ₂	0
q ₁	q ₁	q ₂	1
q ₂	q ₁	q ₂	0

Thus Moore machine $M = (Q, \Sigma, \Delta, \delta, \lambda, q_0)$; where $Q = \{q_0, q_1, q_2\}$, $\Sigma = \{0, 1\}$, $\Delta = \{0, 1\}$.

The transition table shows the δ and λ functions.

Example 1.16.2 Design a Moore and Mealy machine for a binary input sequence such that if it has a substring 101 the machine outputs A if input has substring 110 it outputs B otherwise it outputs C.

SPPU : Dec.-06, May-07, Marks 10

SPPU : Dec.-06, May-07, Marks 10

Solution :

- **Logic**

For designing such a machine we need to take care of two conditions and those are checking 101 and checking 110. If we get 101 the output will be A. If we recognize 110 the output will be B. For other strings the output will be C. We can make a partial design of it as follows.

- **Design**

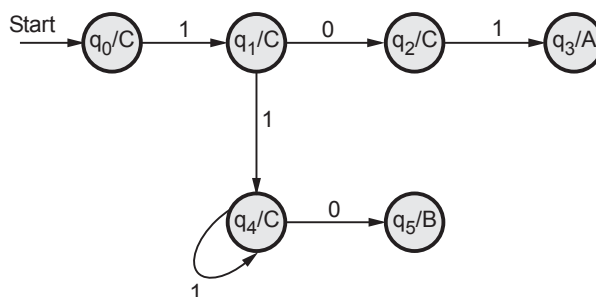


Fig. 1.16.4

Now we will insert the possibilities of 1's and 0's for each state. Then the Moore machine becomes.

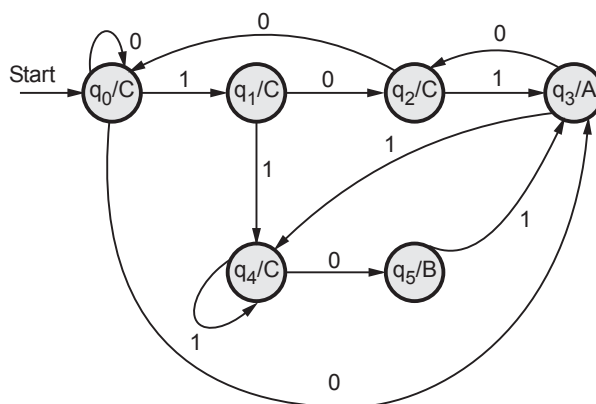


Fig. 1.16.5

Now the Mealy machine can be

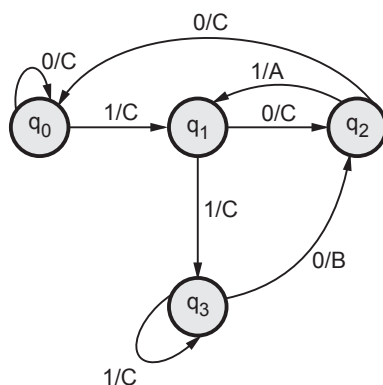


Fig. 1.16.6

Example 1.16.3 Design a Mealy machine to find 2's complement of a given binary number.

SPPU : Dec.-11, Dec.-19, End Sem, Marks 6

Solution :

• **Logic**

For designing 2's complement of a binary number we assume that input is read from LSB to MSB. We will keep the binary number as it is until we read first 1. Keep that 1 as it is then change remaining 1's by 0's and 0's by 1's.

For example,

Let the binary number be

1011

←

read from LSB

Keep the first 1 from LSB as it is and toggle the remaining bits we will get

0101

Thus 2's complement of 1011 is 0101. The required Mealy machine will be -

• **Design**

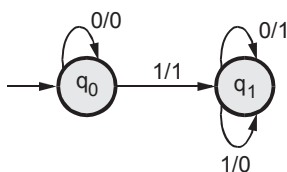


Fig. 1.16.7

Example 1.16.4 Design a Moore machine which will increment the given binary number by 1.

Solution :

• **Logic**

We will read the binary number from LSB one bit at a time. We will replace each 1 by 0 until we get first 0. Once we get first 0 we will replace it by 1 and then keep remaining bits as it is.

For example -

1 0 1 1 ← Read from LSB bit by bit

1 0 1 0

↑

1 0 0 0

↑

1 1 0 0

↑

1 1 0 0

↑

Using this logic we can build a mealy machine

• **Design**

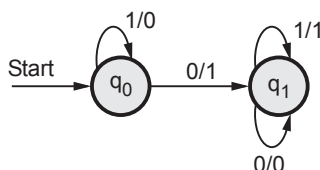


Fig. 1.16.8

Example 1.16.5 Give Moore and Mealy machine for $\Sigma = \{0, 1, 2\}$, print the residue modulo 5 of input treated as a ternary number.

Solution :

• **Logic**

The ternary number is made up of 0,1 and 2. The interpretation of ternary number n can be -

- i) If we write 0 after n then number becomes $3n$.
- ii) If we write 1 after n then number becomes $3n + 1$.
- iii) If we write 2 after n then number becomes $3n + 2$.

For example,

If $n = 4$ then its value is $4 \times 3^0 = 4$.

If we write 0 after 4 i.e.

$$40 = 4 \times 3^1 + 0 \times 3^0 = 12 \quad \text{i.e. } (3 \times 4)$$

If we write 1 after 4 then

$$41 = 4 \times 3^1 + 1 \times 3^0 = 13 \quad \text{i.e. } 3n + 1$$

If we write 2 after 4 we get

$$42 = 4 \times 3^1 + 2 \times 3^0 = 14 \quad \text{i.e. } 3n + 2$$

For residue modulo 5 we will get remainder 0, remainder 1, remainder 2, remainder 3 and remainder 4 values. Then we assume various states for these remainders as -

q_0 - remainder 0 state

q_1 - remainder 1 state

q_2 - remainder 2 state

q_3 - remainder 3 state

q_4 - remainder 4 state

Now consider $n = 4$,

For 4.0 we get decimal value 12 that means $12 \% 5 = 2$ it gives remainder 2.

For 4.1 we get decimal value 13 that means $13 \% 5 = 3$ it gives remainder 3.

Similarly 4.2 given remainder 4.

$n = 4$ itself is remainder 4. Hence we can design,

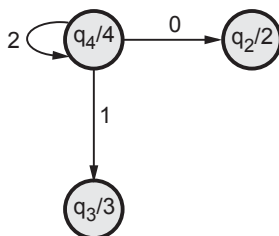


Fig. 1.16.9

• Design

Considering all possible cases we can design Moore machine as,

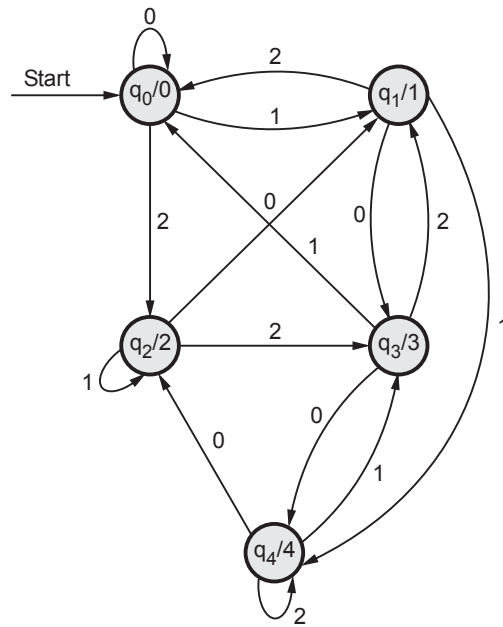


Fig. 1.16.10

Similarly the Mealy machine can be -

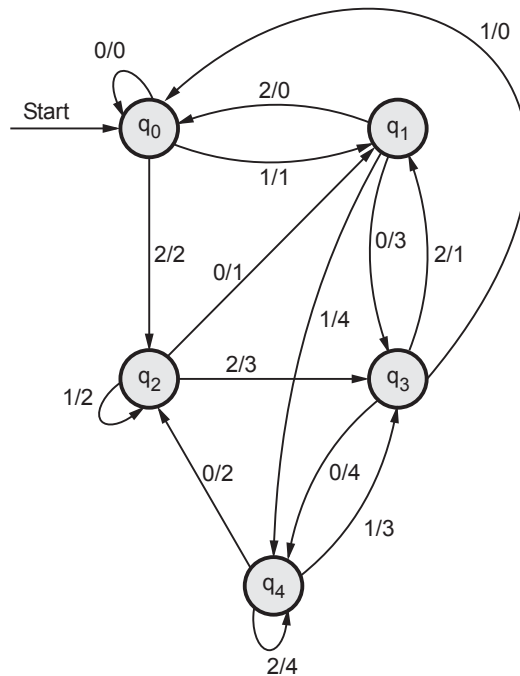


Fig. 1.16.11

Example 1.16.6 Design a Moore machine that will read sequences made up of letters A, E, I, O, U and will give as output the same sequences except that in case where an I directly follows an E, it will be changed to U. Design the Mealy machine for the same.

SPPU : May-13, Marks 8

Solution :

• **Logic**

We will assume a separate state for each alphabet. The output of that state will be corresponding letter. For instance

For alphabet A the state will be q_0 and output of q_0 will be A. For alphabet E the state will be q_1 and output of q_1 will be E. Continuing in this fashion we will have q_0 , q_1 , q_2 , q_3 and q_4 states. The start state will be q_0 we will take care of one thing and that is when I comes immediately after E it will lead to the state q_4 because output of state q_4 is u. With this logic the corresponding Moore machine will be -

• **Design**

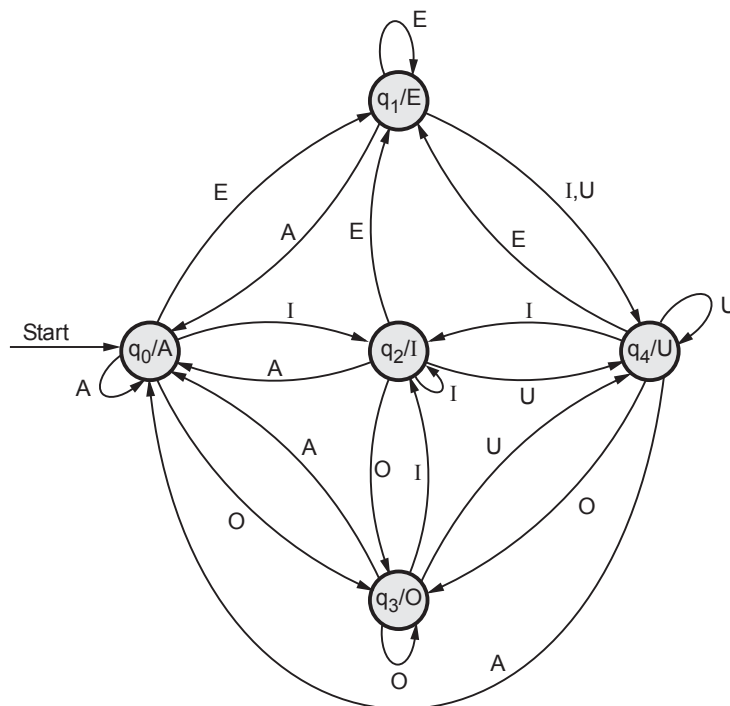


Fig. 1.16.12

Now we will draw the Mealy machine for the same problem.

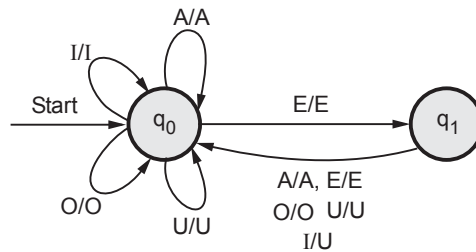


Fig. 1.16.13

Here the Mealy machine has the output along the edge itself we have got only two state. The transition changes from q_0 to q_1 when we get E. The I which is immediately following E will have the output U. Consider the string AIEOAEI will give an output as AIEOAEU.

Example 1.16.7 Construct a Moore machine to determine residue mod 3 for binary number.

Solution :

• **Logic**

This Moore machine is also called remainder 3 tester. In this machine we will get remainder 0, remainder 1 and remainder 2. To interpret the given binary number in its decimal value we consider n as a number if 0 is written after n then its value becomes $2n$. If 1 is written after n then its value becomes $2n + 1$. For instance, if $n = 0$ then its decimal value is 0 then,

$$01 = 2n + 1 = 1 \times 0 + 1 = 1$$

$$011 = 2n + 1 = (1 \times 2) + 1 = 3$$

Consider $n = 1$ its decimal value is 1. After 1 if 0 comes then its value will be

$$10 = 2n = 2 \times 1 = 2$$

If 1 comes after 10 then its value becomes,

$$101 = 2n + 1 = (2 \times 2) + 1 = 5$$

• **Design**

With this logic we can construct a Moore machine with 3 states. q_0 is the start state and is considered as remainder 0 state. q_1 is considered to be remainder 1 state and q_2 is considered as remainder 2 state.

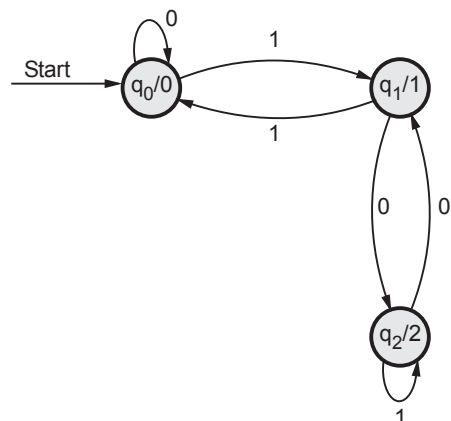


Fig. 1.16.14

Example 1.16.8 Define Mealy machine. Construct a Mealy machine which can output EVEN/ODD if the total number of 1's in the input is even or odd. The input symbols are – 0 and 1.

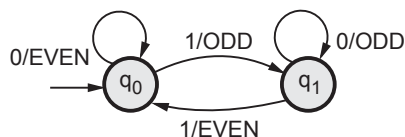
SPPU : Dec-13, Marks 5

Solution :

- **Logic**

The restriction is on number of 1's. There is no restriction on number of 0's. The Mealy machine will then be -

- **Design**



Example 1.16.9 Give Mealy machine for the following processes "For input from $(0+1)^*$, if input ends in 101, output X; if input ends in 110, output Y, otherwise output Z".

SPPU : May-11, 14, Marks 8

Solution :

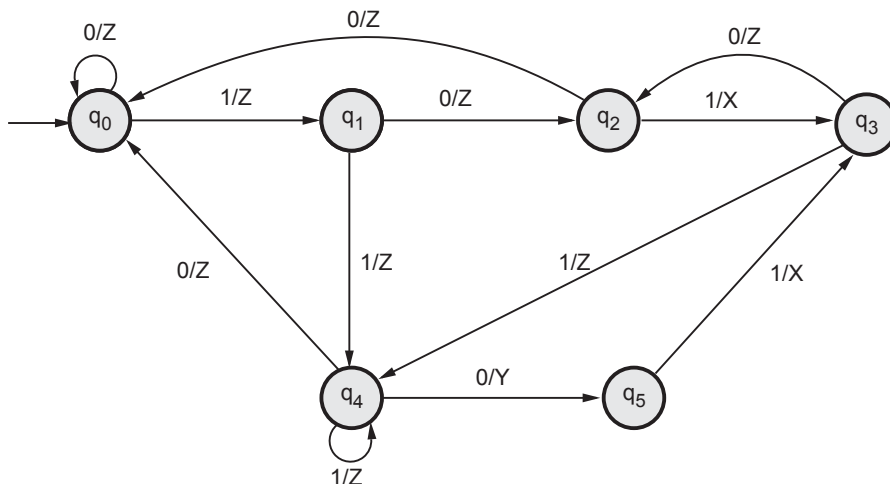


Fig. 1.16.15 Mealy machine

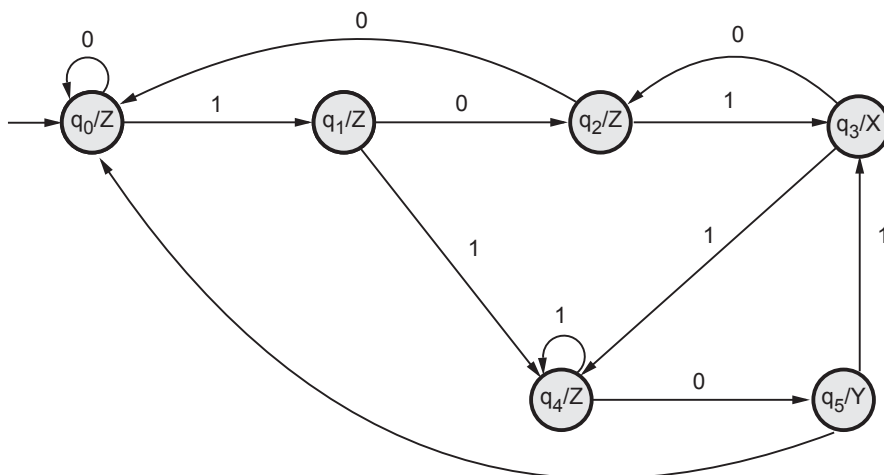


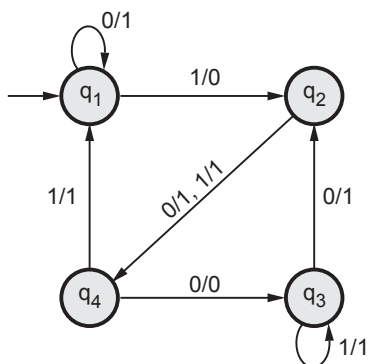
Fig. 1.16.16 Moore machine

Example 1.16.10 Construct a Moore machine equivalent to the Melay machine M defined in the following table with input alphabet, $\Sigma = \{0, 1\}$.

SPPU : Dec.-14, In Sem, Marks 6

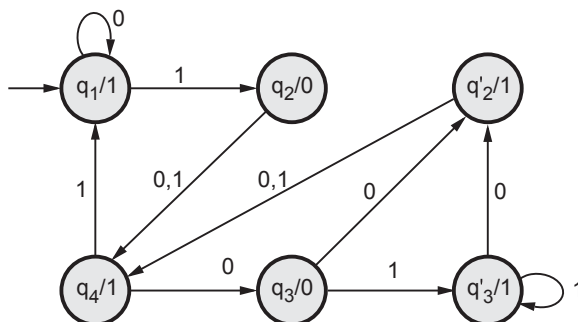
Present state	Next state			
	$\Sigma = 0$		$\Sigma = 1$	
	State	Output	State	Output
$\rightarrow q_1$	q_1	1	q_2	0
q_2	q_4	1	q_4	1
q_3	q_2	1	q_3	1
q_4	q_3	0	q_1	1

Solution : The Melay machine can be drawn as follows -



From this machine we can observe that the edges leading to q_1 and q_4 state have the output 1.

But the state q_3 has output 0 and 1. Similarly edges leading to state q_2 have output 0 and 1. Hence we will split q_2 as q_2 and q'_2 . Similarly we will split q_3 as q_3 and q'_3 . The Moore machine is as follows



Example 1.16.11 Design a Moore machine for computing the 2's complement of binary number. Convert it into its equivalent Mealy machine. **SPPU : Oct.-16, In Sem, Marks 6**

Solution :

- **Logic Applied :** For designing 2's complement of a binary number, we assume that input is read from LSB to MSB. We will keep the binary number as it is until we read first 1. Keep that 1 as it is and then change remaining 1's by 0's and 0's by 1's.
- **Example :** Let the binary number be

1 0 1 1
←
Read from LSB

keep first 1 as it is and then toggle the bits. Hence we get 0101 ← This 2's complemented number.

The Moore machine will be -

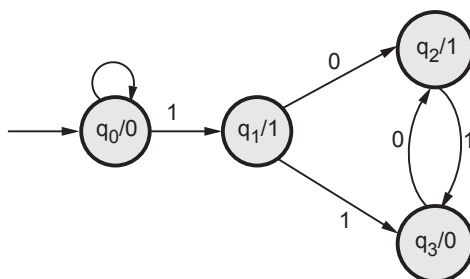


Fig. 1.16.17

- **Equivalent Mealy machine** - Refer ex. 1.16.3.

1.17 Inter-Conversion

SPPU : Dec.-05,11,12,18, May-06,07,12,14,19, Aug.-15,17, Oct.-19, Marks 8

1) Method For conversion of Moore Machine to Mealy Machine

Let $M = (Q, \Sigma, \delta, \lambda, q_0)$ be a Moore machine. The equivalent Mealy machine can be represented by $M' = (Q, \Sigma, \delta, \lambda', q_0)$. The output function λ' can be obtained as -

$$\lambda'(q, a) = \lambda(\delta(q, a))$$

2) Method For conversion of Mealy Machine to Moore Machine

Let $M = (Q, \Sigma, \Delta, \delta, \lambda, q_0)$ be a Mealy machine then there exists a Moore machine M' equivalent to M .

The M' can be given as $M' = (Q \times \Delta, \Sigma, \Delta, \delta', \lambda', [q_0, b_0])$ where b_0 is an output symbol selected from Δ . We will calculate δ' and λ' as follows -

$$\delta'([q, b], a) = [\delta(q, a), \lambda(q, a)]$$

$$\lambda'([q, b]) = b$$

δ' defines the move made by M' on input a .

M' on input a .

λ' defines the output made for the corresponding state q .

Thus we have to calculate δ' and λ' for $q_0, q_1, q_2, \dots, q_n$ on input $a_0, a_1, \dots, a_n$ and should emit the outputs $b_1, b_2, b_3, \dots, b_n$.

Problems Based on Inter-Conversion

Example 1.17.1 The Moore machine to determine residue mod 3 for binary number is given below. Convert it to Mealy equivalent machine.

Q \ Σ	0	1	Output (λ)
q_0	q_0	q_1	0
q_1	q_2	q_0	1
q_2	q_1	q_2	2

Solution : The transition diagram for the given problem can be drawn as -

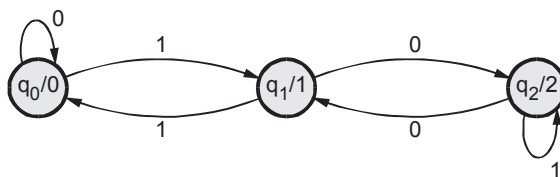


Fig. 1.17.1

The output function λ' can be obtained using following rule,

$$\lambda'(q, a) = \lambda(\delta(q, a))$$

Hence we will obtain output for every transition corresponding to input symbol.

$$\begin{aligned}\lambda'(q_0, 0) &= \lambda(\delta(q_0, 0)) \\ &= \lambda(q_0) \quad \text{i.e. output of } q_0\end{aligned}$$

$$\lambda'(q_0, 0) = 0$$

$$\begin{aligned}\lambda'(q_0, 1) &= \lambda(\delta(q_0, 1)) \\ &= \lambda(q_1) \quad \text{i.e. output of } q_1\end{aligned}$$

$$\lambda'(q_0, 1) = 1$$

$$\begin{aligned}\lambda'(q_1, 0) &= \lambda(\delta(q_1, 0)) \\ &= \lambda(q_2)\end{aligned}$$

$$\lambda'(q_1, 1) = 2$$

$$\begin{aligned}\lambda'(q_1, 1) &= \lambda(\delta(q_1, 1)) \\ &= \lambda(q_0)\end{aligned}$$

$$\lambda'(q_1, 1) = 0$$

$$\begin{aligned}\lambda'(q_2, 0) &= \lambda(\delta(q_2, 0)) \\ &= \lambda(q_1)\end{aligned}$$

$$\lambda'(q_2, 0) = 1$$

$$\begin{aligned}\lambda'(q_2, 1) &= \lambda(\delta(q_2, 1)) \\ &= \lambda(q_2)\end{aligned}$$

$$\lambda'(q_2, 1) = 2$$

Hence the transition table can be drawn as follows -

Q \ Σ	Input 0		Input 1	
	State	O/P	State	O/P
q_0	q_0	0	q_1	1
q_1	q_2	2	q_0	0
q_2	q_1	1	q_2	2

The transition diagram of Mealy machine is,

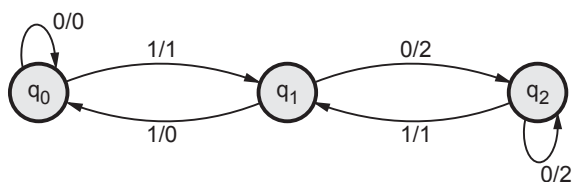


Fig. 1.17.2

The input string 10011 then the output for Mealy machine will be

next state : $q_1 \rightarrow q_2 \rightarrow q_2 \rightarrow q_1 \rightarrow q_0$

output : 1 2 2 1 0

The output sequence for Moore machine will be

Input : 1 0 0 1 1

next state : $q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_2 \rightarrow q_1 \rightarrow q_0$

output : 0 1 2 2 1 0

The length of output sequence is $n+1$ in Moore machine and is n in Mealy machine which is desired.

Example 1.17.2 Convert the following Moore machine into equivalent Mealy machine

$$M = (\{q_0, q_1\}, \{a, b\}, \{0, 1\} \delta, \lambda, q_0).$$

Solution :

δ	a	b	Output (λ)
q_0	q_0	q_1	0
q_1	q_0	q_1	1

The equivalent Mealy machine can be obtained as follows -

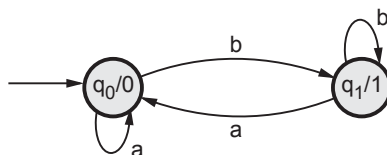


Fig. 1.17.3

$$\begin{aligned}
 \lambda'(q_0, a) &= \lambda(\delta(q_0, a)) \\
 &= \lambda(q_0) \\
 &= 0
 \end{aligned}$$

$$\begin{aligned}
 \lambda'(q_0, b) &= \lambda(\delta(q_0, b)) \\
 &= \lambda(q_1) \\
 &= 1
 \end{aligned}$$

$$\begin{aligned}
 \lambda'(q_1, a) &= \lambda(\delta(q_1, a)) \\
 &= \lambda(q_0) \\
 &= 0
 \end{aligned}$$

$$\begin{aligned}
 \lambda'(q_1, b) &= \lambda(\delta(q_1, b)) \\
 &= \lambda(q_1) \\
 &= 1
 \end{aligned}$$

Hence the transition table can be drawn as follows -

Q \ Σ	Input a		Input b	
	State	O/P	State	O/P
q_0	q_0	0	q_1	1
q_1	q_0	0	q_1	1

The equivalent Mealy machine will be,

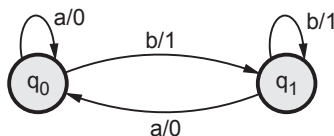


Fig. 1.17.4

Consider the output sequence for Moore machine for input sequence,

a b b a

Then

$$q_0 \xrightarrow{a} q_0 \xrightarrow{b} q_1 \xrightarrow{b} q_1 \xrightarrow{a} q_0$$

0 0 1 1 0

Similarly output sequence for Mealy machine will be

$$q_0 \xrightarrow{a} q_1 \xrightarrow{b} q_1 \xrightarrow{b} q_1 \longrightarrow q_0$$

0 1 1 1 0

We can note that length of Moore machine is $n + 1$ and that of Mealy machine is n .

Example 1.17.3 Consider the Moore machine described by the transition table given below.

Construct corresponding Mealy machine.

SPPU : Dec.-05, Marks 4; May-12, Marks 8

Present state	Next state		Output
	$a = 0$	$a = 1$	
$\rightarrow q_1$	q_1	q_2	0
q_2	q_1	q_3	0
q_3	q_1	q_3	1

Solution : The transition diagram for given Moore machine will be -

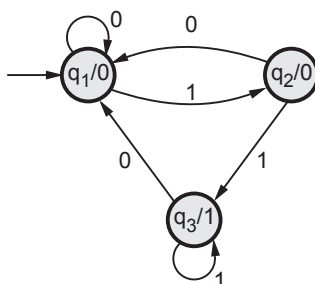


Fig. 1.17.5

The equivalent Mealy machine will be -

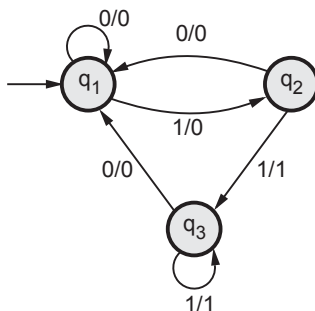


Fig. 1.17.6

Example 1.17.4 Construct a Mealy Machine which is equivalent to the Moore machine given in the following table :

SPPU : Aug.-17, In Sem, Marks 3

Present state	Next state		Output
	$a = 0$	$a = 1$	
$\rightarrow q_0$	q_3	q_1	0
q_1	q_1	q_2	1
q_2	q_2	q_3	0
q_3	q_3	q_0	0

Solution : The transition diagram for the given problem can be drawn as

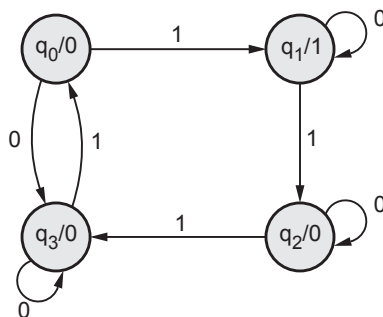


Fig. 1.17.7

The output function λ' can be obtained using following rule,

$$\lambda'(q, 0) = \lambda(\delta(q, 0))$$

Hence we will obtain output function λ' for each transition corresponding to input symbol.

$$\lambda'(q_0, 0) = \lambda(\delta(q_0, 0))$$

$$= \lambda(q_3)$$

$$\lambda'(q_0, 0) = 0$$

$$\lambda'(q_0, 1) = \lambda(\delta(q_0, 1))$$

$$= \lambda(q_1)$$

$$\lambda'(q_0, 1) = 1$$

Thus continuing in this fashion we get following mealy machine.

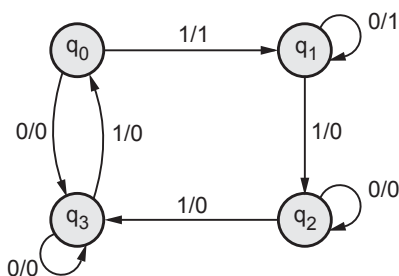


Fig. 1.17.8

Method for conversion of Mealy machine to Moore machine

SPPU : May-06, 10

In Moore machine the output is associated with every state and in Mealy machine the output is given along the edge with input symbol. To convert Moore machine to Mealy machine state output symbols are distributed to input symbol paths. But while

converting Mealy machine to Moore machine we will create a separate state for every new output symbol and according incoming and outgoing edges are distributed.

Let $M = (Q, \Sigma, \Delta, \delta, \lambda, q_0)$ be a Mealy machine then there exists a Moore machine M' equivalent to M .

The M' can be given as $M' = (Q \times \Delta, \Sigma, \Delta, \delta', \lambda', [q_0, b_0])$ where b_0 is an output symbol selected from Δ . We will calculate δ' and λ' as follows -

$$\delta'([q, b], a) = [\delta(q, a), \lambda(q, a)]$$

$$\lambda'([q, b]) = b$$

δ' defines the move made by M' on input a .

M' on input a .

λ' defines the output made for the corresponding state q .

Thus we have to calculate δ' and λ' for $q_0, q_1, q_2, \dots, q_n$ on input $a_0, a_1, \dots, a_n$ and should emit the outputs $b_1, b_2, b_3, \dots, b_n$.

Let us understand this method of conversion with the help of some examples.

Example 1.17.5 Convert the following Mealy machine into equivalent Moore machine.

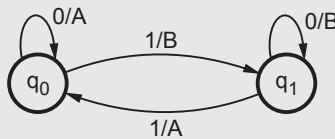


Fig. 1.17.9

Solution : The states for Moore machine will be $[q_0, A], [q_0, B], [q_1, A], [q_1, B]$. Then we will calculate δ' and λ' as follows -

$$\begin{aligned} \delta'([q_0, A], 0) &= [\delta(q_0, 0), \lambda(q_0, 0)] \\ \text{i.e. } &\underbrace{[\text{next state of } q_0 \text{ on input } 0]}_{q_0}, \underbrace{[\text{output of } q_0 \text{ on input } 0]}_A \\ &= [q_0, A] \end{aligned}$$

$$\lambda'([q_0, A]) = A \rightarrow \text{i.e. output of state } q_0.$$

$$\begin{aligned} \delta'([q_1, A], 0) &= [\delta(q_1, 0), \lambda(q_1, 0)] \\ &= [q_1, B] \end{aligned}$$

$$\lambda'([q_1, A]) = A$$

$$\begin{aligned}\delta'([q_1, A], 1) &= [\delta(q_1, 1), \lambda(q_1, 1)] \\ &= [q_0, A]\end{aligned}$$

$$\lambda'([q_1, A]) = A$$

Hence,

	0	1	Output
$[q_0, A]$	$[q_0, A]$	$[q_1, B]$	A
$[q_0, B]$	$[q_0, A]$	$[q_1, B]$	B
$[q_1, A]$	$[q_1, B]$	$[q_0, A]$	A

$$\begin{aligned}\delta'([q_1, B], 0) &= [\delta(q_1, 0), \lambda(q_1, 0)] \\ &= [q_1, B]\end{aligned}$$

$$\lambda'([q_1, B]) = B$$

$$\begin{aligned}\delta'([q_1, B], 1) &= [\delta(q_1, 1), \lambda(q_1, 1)] \\ &= [q_0, B]\end{aligned}$$

$$\lambda'([q_1, B]) = B$$

Finally the transition table will be ,

$$\begin{aligned}\delta'([q_0, A], 1) &= [\delta(q_0, 1), \lambda(q_0, 1)] \\ &= [q_1, B]\end{aligned}$$

$$\lambda'([q_0, A]) = A$$

Hence we can show a partial transition table as,

	0	1	Output
$[q_0, A]$	$[q_0, A]$	$[q_1, B]$	A

Continuing in this fashion we can calculate δ' and λ' as follows -

$$\begin{aligned}\delta'([q_0, B], 0) &= [\delta(q_0, 0), \lambda(q_0, 0)] \\ &= [q_0, A]\end{aligned}$$

$$\lambda'([q_0, B]) = B$$

$$\delta'([q_1, B], 1) = [\delta(q_0, 1), \lambda(q_0, 1)]$$

$$= [q_1, B]$$

$$\lambda'([q_0, B]) = B$$

Hence

	0	1	Output
$[q_0, A]$	$[q_0, A]$	$[q_1, B]$	A
$[q_0, B]$	$[q_0, A]$	$[q_1, B]$	B

State \ I/P	0	1	Output
$[q_0, A]$	$[q_0, A]$	$[q_1, B]$	A
$[q_0, B]$	$[q_0, A]$	$[q_1, B]$	B
$[q_1, A]$	$[q_1, B]$	$[q_0, A]$	A
$[q_1, B]$	$[q_1, B]$	$[q_0, A]$	B

The transition diagram will be,

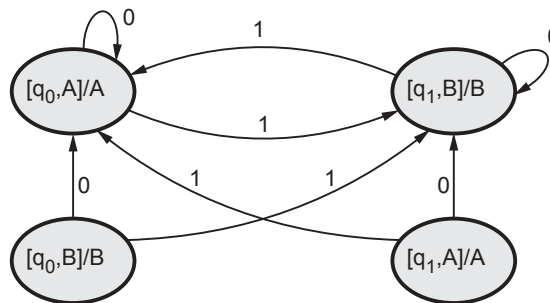


Fig. 1.17.10

Example 1.17.6 Convert the following Mealy machine into equivalent Moore machine.

SPPU : Dec.-18, In Sem, Marks 3

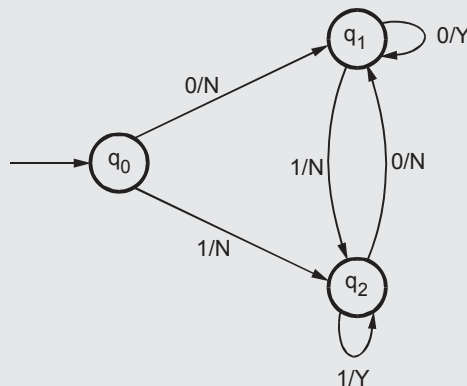


Fig. 1.17.11

Solution : We will write the transition table for given transition graph -

	Input 0	Output	Input 1	Output
q ₀	q ₁	N	q ₂	N
q ₁	q ₁	Y	q ₂	N
q ₂	q ₁	N	q ₂	Y

Now we will find out the states and corresponding outputs for Moore machine. The states for Moore machine will be -

$$[q_0, N], [q_0, Y], [q_1, N], [q_1, Y], [q_2, N], [q_2, Y]$$

Now let us calculate δ' and λ' for all the above given states -

$$\begin{aligned}\delta'([q_0, N], 0) &= [\delta(q_0, 0), \lambda(q_0, 0)] \\ &= [q_1, N]\end{aligned}$$

$$\lambda'([q_0, N]) = N$$

Similarly,

$$\begin{aligned}\delta'([q_0, N], 1) &= [\delta(q_0, 1), \lambda(q_0, 1)] \\ &= [q_2, N]\end{aligned}$$

$$\lambda'([q_0, N]) = N$$

The partial transition table is -

State \ I/P	0	1	Output
[q ₀ , N]	[q ₁ , N]	[q ₂ , N]	N

Now, for remaining states the corresponding transitions and outputs can be obtained as follows -

$$\begin{aligned}\delta'([q_0, Y], 0) &= [\delta(q_0, 0), \lambda(q_0, 0)] \\ &= [q_1, N]N\end{aligned}$$

$$\lambda'([q_0, Y]) = Y$$

$$\begin{aligned}\delta'([q_0, Y], 1) &= [\delta(q_0, 1), \lambda(q_0, 1)] \\ &= [q_2, N]\end{aligned}$$

$$\lambda'([q_0, Y]) = Y$$

$$\begin{aligned}\delta'([q_1, N], 0) &= [\delta(q_1, 0), \lambda(q_1, 0)] \\ &= [q_1, Y]\end{aligned}$$

$$\lambda'([q_1, N]) = N$$

$$\begin{aligned}\delta'([q_1, N], 1) &= [\delta(q_1, 1), \lambda(q_1, 1)] \\ &= [q_2, N]\end{aligned}$$

$$\lambda'([q_1, N]) = N$$

$$\begin{aligned}\delta'([q_1, Y], 0) &= [\delta(q_1, 0), \lambda(q_1, 0)] \\ &= [q_1, Y]\end{aligned}$$

$$\lambda'([q_1, Y]) = Y$$

$$\begin{aligned}\delta'([q_2, N], 0) &= [\delta(q_2, 0), \lambda(q_2, 0)] \\ &= [q_1, N]\end{aligned}$$

$$\lambda'([q_2, N]) = N$$

$$\begin{aligned}\delta'([q_2, N], 1) &= [\delta(q_2, 1), \lambda(q_2, 1)] \\ &= [q_2, Y]\end{aligned}$$

$$\lambda'([q_2, N]) = N$$

$$\begin{aligned}\delta'([q_1, Y], 1) &= [\delta(q_1, 1), \lambda(q_1, 1)] \\ &= [q_2, N]\end{aligned}$$

$$\lambda'([q_1, Y]) = Y$$

$$\begin{aligned}\delta'([q_2, Y], 0) &= [\delta(q_2, 0), \lambda(q_2, 0)] \\ &= [q_1, N]\end{aligned}$$

$$\lambda'([q_2, Y]) = Y$$

$$\begin{aligned}\delta'([q_2, Y], 1) &= [\delta(q_2, 1), \lambda(q_2, 1)] \\ &= [q_2, Y]\end{aligned}$$

$$\lambda'([q_2, Y]) = Y$$

The transition table can be drawn as -

State \ Σ	0	1	Output
$[q_0, N]$	$[q_1, N]$	$[q_2, N]$	N
$[q_0, Y]$	$[q_1, N]$	$[q_2, N]$	Y
$[q_1, N]$	$[q_1, Y]$	$[q_2, N]$	N
$[q_1, Y]$	$[q_1, Y]$	$[q_2, N]$	Y
$[q_2, N]$	$[q_1, N]$	$[q_2, Y]$	N
$[q_2, Y]$	$[q_1, N]$	$[q_2, Y]$	Y

Example 1.17.7 Construct a Moore machine equivalent to the Mealy machine M given below.

δ is

SPPU : Dec.-11, Marks 6

		$a = 0$	$a = 1$
q_1	q_1	1	q_2 0
q_2	q_4	1	q_4 1
q_3	q_2	1	q_3 1
q_4	q_3	0	q_1 1

Solution : We will first design the transition diagram from given transition table.

The transition table for Moore machine will be

	0	1	Output
$[q_1]$	$[q_1]$	$[q_2]$	1
$[q_2]$	$[q_4]$	$[q_4]$	0 and 1
$[q_3]$	$[q_2]$	$[q_3]$	0 and 1
$[q_4]$	$[q_3]$	$[q_1]$	1

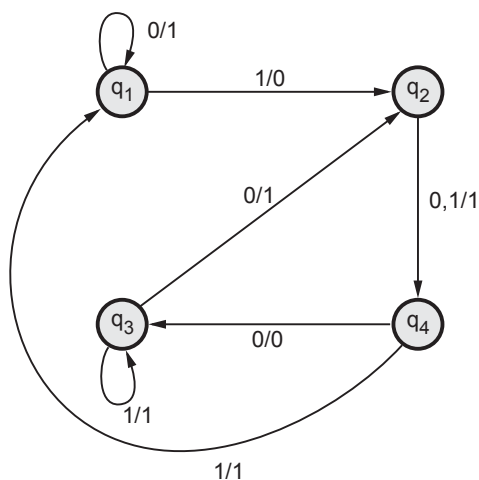


Fig. 1.17.12

The Moore machine will be

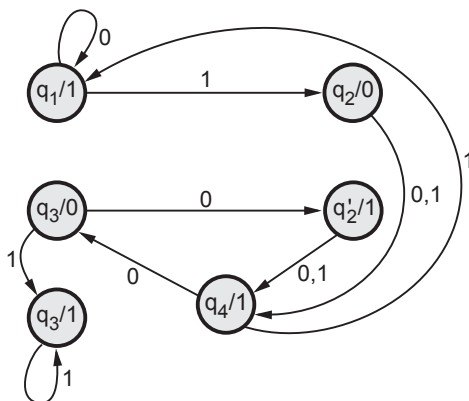


Fig. 1.17.13

Example 1.17.8 Construct Mealy machine equivalent to Moore machine given as :

SPPU : Dec.-12, Marks 6

Present state	Next state		Output
	$a = 0$	$a = 1$	
$\rightarrow q_0$	q_1	q_2	1
q_1	q_3	q_2	0
q_2	q_2	q_1	1
q_3	q_0	q_3	1

Solution : Equivalent Mealy machine can be obtained as follows :

$$\lambda'(q_0, 0) = \lambda(\delta(q_0, 0)) = \lambda(q_1) = 0$$

$$\lambda'(q_0, 1) = \lambda(\delta(q_0, 1)) = \lambda(q_2) = 1$$

$$\lambda'(q_1, 0) = \lambda(\delta(q_1, 0)) = \lambda(q_3) = 1$$

$$\lambda'(q_1, 1) = \lambda(\delta(q_1, 1)) = \lambda(q_2) = 1$$

$$\lambda'(q_2, 0) = \lambda(\delta(q_2, 0)) = \lambda(q_2) = 1$$

$$\lambda'(q_2, 1) = \lambda(\delta(q_2, 1)) = \lambda(q_1) = 0$$

$$\lambda'(q_3, 0) = \lambda(\delta(q_3, 0)) = \lambda(q_0) = 1$$

$$\lambda'(q_3, 1) = \lambda(\delta(q_3, 1)) = \lambda(q_3) = 1$$

Present state	Next state			
	$a = 0$	Output	$a = 1$	Output
$\rightarrow q_0$	q_1	0	q_2	1
q_1	q_3	1	q_2	1
q_2	q_2	1	q_1	0
q_3	q_0	1	q_3	1

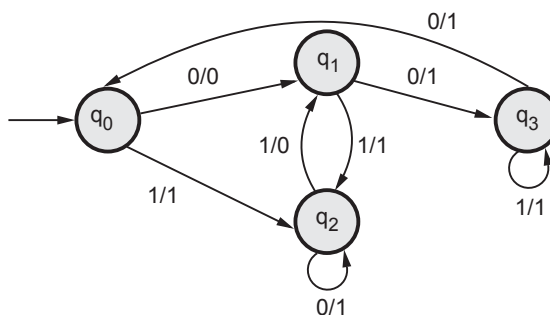


Fig. 1.17.14

Example 1.17.9 Convert the following Mealy machine to Moore machine.

SPPU : May-07, Marks 4

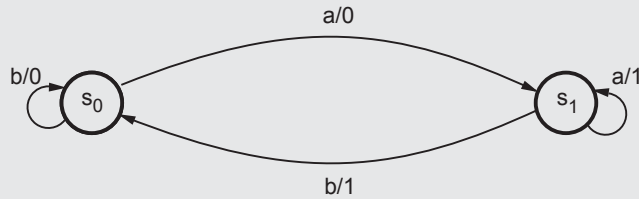


Fig. 1.17.15

Solution : If we observe state so then we notice that state so gets two incoming edges one for input b and output 0 and the other with input b and output 1. Similarly state S_1 gets two incoming edges with output 0 and output 1. Hence we split up the states as.

$S_0 \Rightarrow S_0$ and S'_0 with output 0 and 1 respectively.

$S_1 \Rightarrow S_1$ and S'_1 with output 0 and 1 respectively.

The Moore machine will then be -

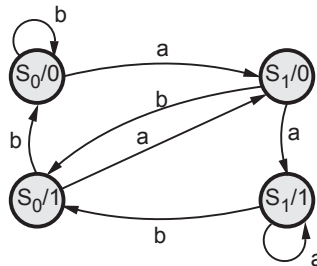


Fig. 1.17.16

Example 1.17.10 Convert the following Mealy machine to More machine.

SPPU : Oct.-19, In Sem, Marks 4

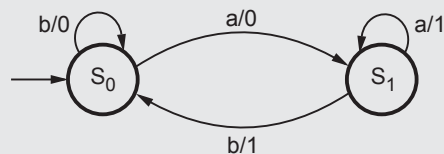


Fig. 1.17.17

Solution : The states for Moore machine will be $[S_0, 0]$, $[S_0, 1]$, $[S_1, 0]$, $[S_1, 1]$. Then we will be calculate δ' and λ' as follows :

$$\begin{aligned}
 1) \quad \delta'([S_0, 0], a) &= [\delta(S_0, a), \lambda(S_0, a)] \\
 &= [S_1, 0]
 \end{aligned}$$

- $$\lambda'([S_0, 0]) = 0$$
- 2) $\delta'((S_0, 0), b) = [\delta(S_0, b), \lambda(S_0, b)]$
 $= [S_0, 0]$
- $$\lambda'([S_0, 0]) = 0$$
- $$\delta'((S_1, 0), a) = [\delta(S_1, a), \lambda(S_1, a)]$$
- $$= [S_1, 1]$$
- $$\lambda'([S_1, 0]) = 0$$
- 3) $\delta'((S_1, 0), b) = [\delta(S_1, b), \lambda(S_1, b)]$
 $= [S_0, 1]$
- $$\lambda'([S_1, 0]) = 0$$
- 4) $\delta'((S_0, 1), a) = [\delta(S_0, a), \lambda(S_0, a)]$
 $= [S_1, 0]$
- $$\lambda'([S_0, 1]) = 1$$
- 5) $\delta'((S_0, 1), b) = [\delta(S_0, b), \lambda(S_0, b)]$
 $= [S_0, 0]$
- $$\lambda'([S_0, 1]) = 1$$
- 6) $\delta'([S_1, 1], a) = [\delta(S_1, a), \lambda(S_1, a)]$
 $= [S_1, 1]$
- $$\lambda'([S_1, 1]) = 1$$
- 7) $\delta'([S_1, 1], b) = [\delta(S_1, b), \lambda(S_1, b)]$
 $= [S_0, 1]$
- $$\lambda'([S_1, 1]) = 1$$

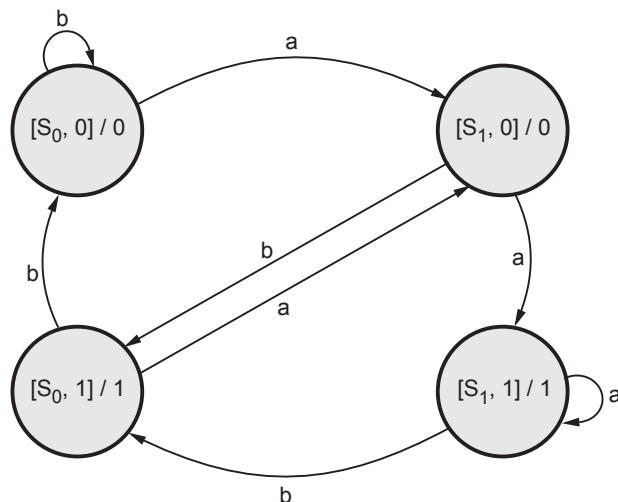


Fig. 1.7.17 (a)

The Moore machine will be :

Review Questions

1. Define Moore and Mealy machine and how the equivalence of Moore and Mealy machine can be established with example. **SPPU : May-14, Marks 6**
2. Give formal definition for a Moore machine with a suitable example
SPPU : Aug.-15, In Sem, May-19, End Sem, Marks 2
3. Compare Moore and Mealy machine.
SPPU : Dec.-18, In Sem, Marks 3, Oct.-19, In Sem, Marks 2

1.18 Case Study

FSM for Vending Machine

The states of coffee vending machine can be represented using finite State machine. Here is an illustrative vending machine with following assumptions

- When ₹ 15 are deposited then only release coffee.
- There will be single coin slot for ₹ 10 and ₹ 5

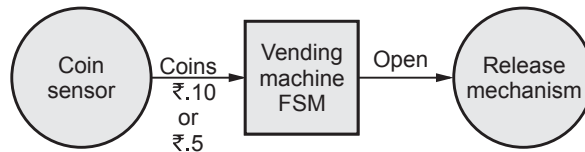


Fig. 1.18.1 Abstract model for vending machine

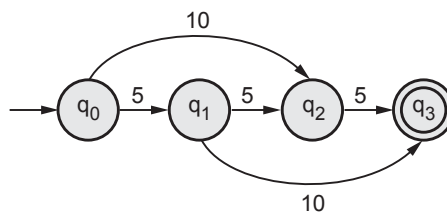
- No Change

The abstract representation of the machine will be

There are various cases that will lead to the state of release mechanism.

Case 1 : 3 coins of ₹ 5

Case 2 : 2 coins - firstly ₹ 5 coin and then ₹ 10 coin.



Case 3 : 2 coins - firstly ₹ 10 coin and then ₹ 5 coin.

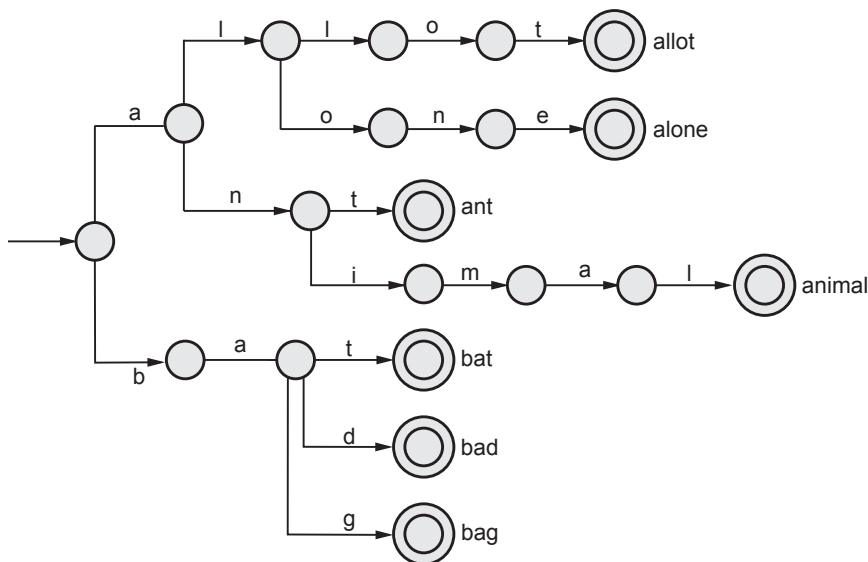
The FSM can be as follows

Her q_0 is the initial state with no coin inserted. The value on each of edge of FSM represents the value of coin. The state q_3 is a final state. When machine reaches to this state, the coffee will be released from the vending machine.

FSM in Spell Check

- Finite State machines can be used for simple spell checking.
- In this implementation, one or more words are provided by the user and those words would either be confirmed or rejected as being valid in that language.
- At the basic level, the finite state automata is generated for each of the valid string in the language.
- The common prefixes or common suffixes are used to recognize the words. For example

The use of finite state machine improves the performance of pattern matching procedure.



All the final states in above FSM indicate the correct spellings and all the intermediate states represent incomplete or incorrect spelling of a particular word.

1.19 Fill in the Blanks

- Q.1** DFA stands for ____.
- Q.2** The language generated by DFA is ____.
- Q.3** Power of DFA and NFA is ____.
- Q.4** The ____ character is used to represent empty null string in non deterministic finite automata.
- Q.5** $\{L = a^p | P \text{ is prime}\}$ is a ____ language.
- Q.6** Finite automata can recognize only one type of language and that is ____.
- Q.7** Can DFA simulate NFA ?
- Q.8** How should be the recognizing capacity of NFA and DFA same or different ?
- Q.9** An automation is ____ device and grammar is a ____ device.
- Q.10** Finite state machine ____ recognize panlindrome (can/can't).
- Q.11** Removing ____ states from a machine M does not affect the language it accepts.
- Q.12** Let maximum number of states in DFA=128, then its equivalent NFA has ____ states.
- Hint :** The maximum number of states in DFA= 2^{n+1} where n is number of states in NFA.

1.20 Multiple Choice Questions

- Q.1** For a DFA accepting binary numbers whose decimal equivalent is divisible by 4, what are all the possible remainders ?
- | | |
|----------------------------------|------------------------------------|
| ☐ a 0 | ☐ b 0,2 |
| ☐ c 0,2,4 | ☐ d 0,1,2,3 |
- Q.2** The number of states to accept string "ba" are ____.
- | | |
|------------------------------|------------------------------|
| ☐ a 1 | ☐ b 2 |
| ☐ c 3 | ☐ d 4 |
- Q.3** Language of finite automata is ____.
- | | |
|-----------------------------------|-----------------------------------|
| ☐ a Type 0 | ☐ b Type 1 |
| ☐ c Type 2 | ☐ d Type 3 |
- Q.4** There are ____ number of stacks used in finite automata.
- | | |
|------------------------------|------------------------------|
| ☐ a 0 | ☐ b 1 |
| ☐ c 2 | ☐ d 3 |

Q.5 Number of final state require to accept ϕ in minimal finite automata.

- | | |
|------------------------------|---|
| ☐ a 1 | ☐ b 2 |
| ☐ c 3 | ☐ d No final state is required |

Q.6 How many DFA's exists with two states over input alphabet $\{a,b\}$?

- | | |
|-------------------------------|-------------------------------|
| ☐ a 2 | ☐ b 16 |
| ☐ c 64 | ☐ d 32 |

Hint : Number of DFA's = $2^n \cdot n(2^n)$.

Q.7 An FSM with ____.

- | |
|---|
| ☐ a one stack is powerful than two stack |
| ☐ b one stack is powerful than two stack |
| ☐ c both are of equal power |
| ☐ d none of these |

Q.8 Every NFA accepts a ____.

- | | |
|---|--|
| ☐ a string | ☐ b function |
| ☐ c regular language | ☐ d context free language |

Q.9 $N = (Q, \epsilon, \delta, S, F)$ represents a ____.

- | | |
|--------------------------------|---|
| ☐ a NFA | ☐ b PDA |
| ☐ c LDA | ☐ d Turing Machine |

Q.10 An NFA's transition function returns ____.

- | | |
|--|----------------------------------|
| ☐ a boolean value | ☐ b state |
| ☐ c set of states | ☐ d edge |

Q.11 Any given transition graph has equivalent ____.

- | | |
|---|---|
| ☐ a regular expression | ☐ b DFA |
| ☐ c NFA | ☐ d all of these |

Answer Keys for Fill in the Blanks :

Q.1	Deterministic Finite Automate	Q.2	Regular language
Q.3	same	Q.4	ϵ or $\wedge$
Q.5	non regular	Q.6	Regular language
Q.7	Yes	Q.8	Same
Q.9	cognitive, generative	Q.10	can't
Q.11	dead or unreachable	Q.12	6

Answer Keys for Multiple Choice Questions :

Q.1	d	Q.2	c	Q.3	d	Q.4	a
Q.5	d	Q.6	c	Q.7	b	Q.8	c
Q.9	a	Q.10	c	Q.11	d		

